



Università
di Catania

UNIVERSITY OF CATANIA

DEPARTMENT OF MATHEMATICS AND COMPUTER SCIENCE

A MASTER OF SCIENCE DEGREE IN COMPUTER SCIENCE.

Alfio Spoto

Blockchain and Cryptocurrencies

LECTURE NOTES

Professor: Dario Catalano
Professor: Mario Di Raimondo

Academic Year 2024 - 2025

Contents

II	Lezioni di Laboratorio	4
1	Lezione 1	5
1.1	Transazioni Bitcoin: Modelli di Transazione	5
1.1.1	Modello Account-Based	5
1.1.2	Modello Transaction-Based (UTXO)	5
1.2	Anatomia di una Transazione Bitcoin	6
1.2.1	Struttura dei Metadati	7
1.2.2	Struttura degli Input	7
1.2.3	Struttura degli Output	7
1.3	Bitcoin Script e Meccanismi di Verifica	8
1.3.1	Natura e Caratteristiche di Bitcoin Script	8
1.3.2	Meccanismo Pay-to-PubKeyHash (P2PKH)	9
1.3.3	Applicazioni Avanzate di Bitcoin Script	9
2	Lezione 2	11
2.1	Struttura della Blockchain: Blocchi e Loro Organizzazione	11
2.1.1	Struttura del Blocco	11
2.1.2	Albero di Merkle: Struttura ed Efficienza	12
2.1.3	Catena di Blocchi e Immutabilità	13
2.2	Transazioni Speciali: Coinbase	13
2.2.1	Creazione di Nuovi Bitcoin	13
2.2.2	Schema di Ricompensa e Halving	14
2.2.3	Aspetti Storici e Culturali	15
3	Lezione 3	16
3.1	Rete Bitcoin: Architettura di Rete	16
3.1.1	Caratteristiche della Rete P2P	16
3.1.2	Meccanismi di Discovery	16
3.2	Propagazione di Transazioni	17
3.2.1	Protocollo di Gossip	18
3.2.2	Transaction Pool	18
3.3	Propagazione dei Blocchi	19
3.3.1	Criteri di Validazione e Propagazione	19
3.3.2	Statistiche di Propagazione	20
3.4	Tipologie di Nodi	20
3.4.1	Full Nodes	20
3.4.2	Nodi Lightweight (SPV)	21

3.4.3	Distribuzione Geografica e Statistica	21
4	Lezione 4	22
4.1	Limitazioni e Scalabilità di Bitcoin: Vincoli di Progettazione	22
4.1.1	Parametri Fondamentali	22
4.1.2	Primitive Crittografiche	23
4.2	Meccanismi di Evoluzione del Protocollo	23
4.2.1	Hard Fork vs. Soft Fork	23
4.3	Fondamenti della Proprietà Crittografica	24
4.3.1	Relazione tra Chiavi e Identità	24
4.4	Wallet: Funzionalità e Tipologie	25
4.4.1	Caratteristiche Fondamentali	25
4.4.2	Indirizzi e Pseudonimia	25
4.5	Strategia di Storage: Hot vs. Cold	26
4.5.1	Hot Storage	26
4.5.2	Cold Storage	26
4.5.3	Flussi Operativi Ibridi	27
4.6	Wallet Deterministici Gerarchici (HD Wallet)	28
4.6.1	Separazione delle Funzionalità e Meccanismo di Derivazione . . .	29
4.7	Fondamenti di ECDSA	30
4.7.1	Parametri e Componenti	30
4.7.2	Processo di Firma	30
4.7.3	Processo di Verifica	31
4.8	Implementazioni di Cold Storage	31
4.8.1	Considerazioni sulla Sicurezza delle Passphrase	32
4.9	Secret Sharing e Threshold Cryptography	32
4.9.1	Principi del Secret Sharing	33
4.9.2	Threshold Secret Sharing	33
4.9.3	Threshold Signatures	34
4.10	Bitcoin Exchanges	34
4.10.1	Proof of Reserves	35
5	Lezione 5	36
5.1	Il Processo di Mining	36
5.1.1	Attività del Miner	36
5.1.2	Difficoltà di Mining	36
5.1.3	Aggiustamento Dinamico della Difficoltà	37
5.2	Tecniche di Mining	37
5.2.1	Mining con CPU	37
5.2.2	Mining con GPU	38
5.2.3	Mining con FPGA	38
5.2.4	Mining con ASIC	39
5.3	Mining Pool	39
5.3.1	Principio di Funzionamento	39
5.3.2	Modelli di Distribuzione della Ricompensa	39
6	Capitolo 6	41
6.1	Dal Contratto Tradizionale allo Smart Contract	41
6.1.1	Paradigmi Contrattuali a Confronto	41

6.1.2	Limiti di Bitcoin Script ed Evoluzione Necessaria	42
6.1.3	NameCoin: Un Caso di Studio Emblematico	42
6.2	La Piattaforma Ethereum: Concetti Fondanti	43
6.2.1	Visione e Applicazioni	43
6.2.2	Architettura Concettuale: Ethereum vs Bitcoin	44
6.2.3	La Valuta Nativa: Ether (ETH)	46
6.2.4	Proof of Stake	46
6.2.5	Mining e Validazione: Dal Proof-of-Work al Proof-of-Stake	47
7	Capitolo 7	48
7.1	Struttura degli Account e Gestione dello Stato	48
7.1.1	Tipologia e Proprietà degli Account	48
7.1.2	Merkle Patricia Trie: Una Struttura Dati Ibrida	49
7.1.3	Organizzazione Gerarchica dello Stato	50
7.2	Transazioni, Messaggi e Costruzione dei Blocchi	52
7.2.1	Anatomia delle Transazioni	52
7.2.2	Messaggi e Composabilità	53
7.2.3	Processo di Costruzione dei Blocchi	55
7.2.4	Struttura dei Blocchi	56
7.3	Il Sistema del Gas: Economia Computazionale	57
7.3.1	Problema dell'Halting e Soluzioni	57
7.3.2	Meccanismo Legacy del Gas	58
7.3.3	EIP-1559: Rivoluzione nel Mechanism Design	59
8	Capitolo 8: Esempi di Transazioni Reali e Implicazioni	61
8.1	Anatomia di Transazioni Concrete	61
8.1.1	Trasferimento di Fondi	61
8.1.2	Interazione con Contratti (ERC-20)	62
8.1.3	Transazioni Fallite (Out-of-Gas)	62
8.2	Vulnerabilità ed Errori Celebri	62
8.2.1	The DAO Hack (2016)	62
8.2.2	Altri Casi Notevoli	63
8.3	Implicazioni Filosofiche e Legali	63
8.3.1	Code is Law?	64
8.3.2	Responsabilità Giuridica	64
8.3.3	Evoluzioni di Governance	64

Part II

Lezioni di Laboratorio

Chapter 1

Lezione 1

1.1 Transazioni Bitcoin: Modelli di Transazione

Le transazioni in Bitcoin possono essere concettualizzate secondo due paradigmi distinti: il modello basato su account e il modello basato su transazioni (UTXO). Questi modelli differiscono significativamente nella loro struttura, efficienza e implicazioni per la verificabilità e la sicurezza del sistema.

1.1.1 Modello Account-Based

Il modello basato su account risulta familiare a chi conosce i sistemi bancari tradizionali. In questo paradigma, ogni utente possiede un account identificato univocamente con un saldo associato che viene aggiornato sequenzialmente con ogni transazione. Le operazioni sono concettualmente semplici: quando Alice trasferisce bitcoin a Bob, il suo saldo diminuisce e quello di Bob aumenta di conseguenza.

Questo modello, tuttavia, presenta limitazioni significative nell'ambito delle criptovalute decentralizzate. La verifica delle transazioni risulta inefficiente poiché richiede la ricostruzione dell'intero stato dell'account attraverso la cronologia completa delle transazioni pregresse, generando una complessità computazionale che cresce linearmente con il numero di operazioni. Inoltre, gli aggiornamenti simultanei necessiterebbero di meccanismi di locking degli account difficilmente implementabili in un sistema decentralizzato, compromettendo la scalabilità del sistema.

1.1.2 Modello Transaction-Based (UTXO)

Bitcoin implementa invece un modello basato su Unspent Transaction Outputs (UTXO), che considera il denaro digitale come una collezione discreta di "monete" che possono essere spese solo integralmente. Questo approccio rappresenta un cambio paradigmatico rispetto ai sistemi finanziari tradizionali.

Nel modello UTXO, ogni transazione consuma uno o più output non spesi (input) e genera nuovi output che potranno essere utilizzati come input per transazioni successive. Questo meccanismo crea una transizione di stato atomica e facilmente verificabile. Una caratteristica fondamentale è l'assenza di "conti" persistenti: esistono solo output di transazioni che possono essere utilizzati come input per transazioni successive.

La validazione di una transazione UTXO richiede solamente la conferma che gli UTXO utilizzati come input esistano e non siano già stati spesi, senza necessità di ricostruire l'intero stato del sistema. Questo approccio offre vantaggi significativi in termini di efficienza di verifica e parallelizzazione delle operazioni.

Una transazione UTXO viene validata attraverso quattro controlli fondamentali:

1. Verifica che ogni input referenzi un UTXO esistente nella blockchain.
2. Controllo che tali UTXO non siano già stati consumati in transazioni precedenti (prevenzione del double-spending).
3. Validazione delle firme crittografiche associate a ciascun input, garantendo che solo i legittimi proprietari possano spendere i fondi.
4. Verifica che la somma degli input sia maggiore o uguale alla somma degli output, con la differenza eventualmente destinata come commissione per i miners.

Il modello UTXO supporta naturalmente funzionalità avanzate come il consolidamento di valori (combinazione di UTXO multipli in un'unica transazione), pagamenti congiunti che richiedono l'approvazione di più parti, e migliore privacy grazie alla generazione di output multipli indipendenti in una stessa transazione.

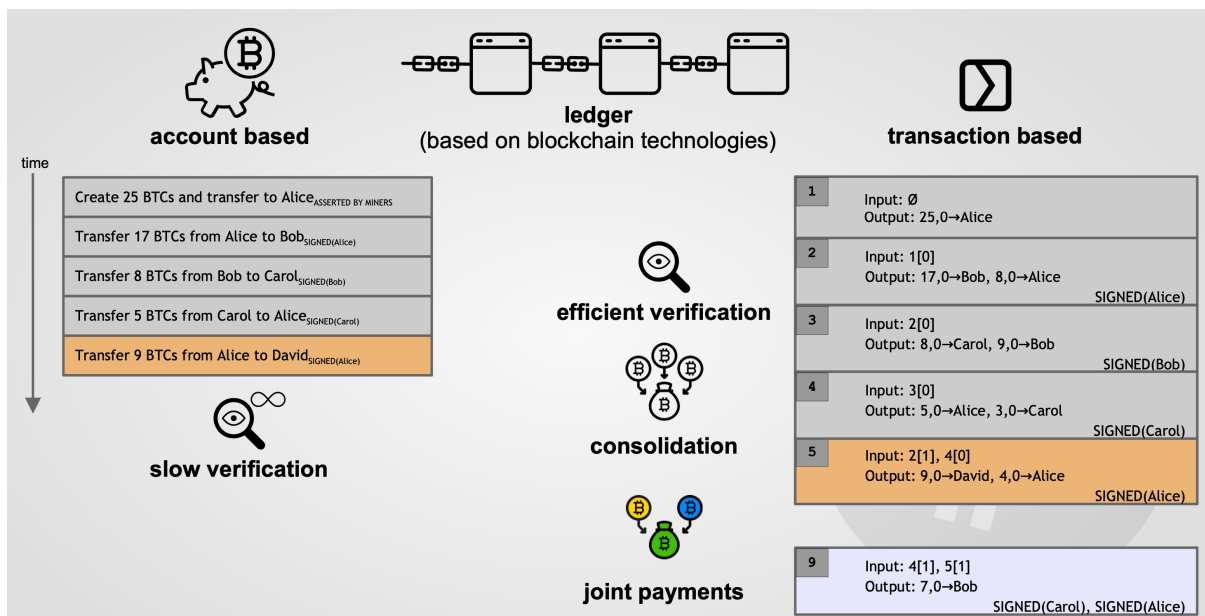


Figure 1.1: Confronto tra i modelli di transazione account-based e transaction-based (UTXO) in Bitcoin, con esempi di transazioni e vantaggi relativi a consolidamento, pagamenti congiunti e verificabilità.

1.2 Anatomia di una Transazione Bitcoin

Una transazione Bitcoin è una struttura dati composta da tre elementi principali: meta-dati, input e output. Esaminare questa struttura fornisce una comprensione dettagliata del funzionamento del sistema di pagamento.

1.2.1 Struttura dei Metadati

I metadati di una transazione contengono informazioni essenziali per l'identificazione e l'elaborazione dell'operazione:

- **hash**: Identificativo univoco della transazione, calcolato attraverso la funzione hash SHA-256 applicata all'intera transazione.
- **ver**: Versione del protocollo utilizzato, che determina le regole di interpretazione e validazione.
- **vin_sz/vout_sz**: Numero di input e output contenuti nella transazione.
- **lock_time**: Timestamp che può essere utilizzato per ritardare l'elaborazione della transazione fino a un determinato momento.
- **size**: Dimensione in byte dell'intera transazione, rilevante per il calcolo delle commissioni.

1.2.2 Struttura degli Input

Gli input di una transazione rappresentano i fondi che vengono consumati. Ogni input contiene:

- **prev_out**: Riferimento all'output di una precedente transazione che viene consumato, identificato dall'hash della transazione originaria e dall'indice dell'output specifico.
- **scriptSig**: Script di firma (o "script di sblocco") che fornisce i dati necessari per soddisfare le condizioni di spendibilità dell'output referenziato. Nella forma più comune (P2PKH - Pay-to-Public-Key-Hash), contiene:
 - La firma digitale creata con la chiave privata del proprietario
 - La chiave pubblica completa del proprietario

1.2.3 Struttura degli Output

Gli output rappresentano i nuovi fondi creati dalla transazione e destinati ai beneficiari. Ogni output contiene:

- **value**: Ammontare di bitcoin assegnato a questo output, espresso in satoshi (1 bitcoin = 100,000,000 satoshi).
- **scriptPubKey**: Script pubblico (o "script di blocco") che definisce le condizioni necessarie per spendere l'output. Nella forma P2PKH, questo script:
 - Contiene l'hash della chiave pubblica del destinatario (non la chiave pubblica completa)
 - Verifica che la firma fornita in un futuro input sia valida per quella chiave pubblica
 - Verifica che l'hash della chiave pubblica fornita corrisponda all'hash memorizzato

L'interazione tra scriptSig e scriptPubKey è fondamentale: quando una transazione viene verificata, il sistema Bitcoin esegue prima lo scriptSig dell'input e poi lo scriptPubKey dell'output referenziato. Se l'esecuzione combinata termina con successo, l'input è valido.



Figure 1.2: Struttura dettagliata di una transazione Bitcoin, con evidenziazione dei suoi componenti principali: metadati, input e output.

1.3 Bitcoin Script e Meccanismi di Verifica

Il sistema di scripting di Bitcoin costituisce un elemento fondamentale dell'architettura di validazione delle transazioni, fornendo flessibilità e sicurezza attraverso un linguaggio di programmazione semplice ma espressivo.

1.3.1 Natura e Caratteristiche di Bitcoin Script

Bitcoin Script è un linguaggio di programmazione stack-based, appositamente progettato per essere:

- **Non Turing-completo:** Privo di cicli e ricorsione per prevenire attacchi di denial-of-service.
- **Deterministico:** Ogni esecuzione dello stesso script produce invariabilmente lo stesso risultato.
- **Minimalista:** Supporta solo le operazioni essenziali per la verifica crittografica e la manipolazione dello stack.

Questa semplicità strutturale conferisce al linguaggio proprietà vantaggiose per un sistema finanziario distribuito, garantendo prevedibilità (ogni script ha un risultato determinato esclusivamente dal suo contenuto), verificabilità (la validità di uno script può essere verificata indipendentemente da ogni nodo della rete) ed efficienza (l'esecuzione richiede risorse computazionali limitate e prevedibili).

1.3.2 Meccanismo Pay-to-PubKeyHash (P2PKH)

Il meccanismo standard di trasferimento bitcoin, denominato Pay-to-PubKeyHash (P2PKH), implementa una logica di verifica basata sull'associazione tra chiavi pubbliche e firme digitali:

- **scriptSig**: Fornisce i dati necessari per soddisfare le condizioni del scriptPubKey e ha il seguente formato:

`<Firma> <PubKey>`

dove `<Firma>` è una firma digitale creata con la chiave privata corrispondente alla chiave pubblica `<PubKey>`.

- **scriptPubKey**: Definisce le condizioni di spendibilità e ha il seguente formato:

`OP_DUP OP_HASH160 <PubKeyHash> OP_EQUALVERIFY OP_CHECKSIG`

dove `<PubKeyHash>` è l'hash a 160-bit della chiave pubblica del destinatario.

Il processo di verifica avviene attraverso l'esecuzione concatenata di entrambi gli script sulla macchina virtuale di Bitcoin. Prima viene eseguito il scriptSig, posizionando nello stack la firma e la chiave pubblica del proponente. Successivamente viene eseguito lo scriptPubKey, che:

1. Duplica la chiave pubblica in cima allo stack (`OP_DUP`)
2. Calcola l'hash della chiave pubblica duplicata (`OP_HASH160`)
3. Confronta l'hash calcolato con il `<PubKeyHash>` memorizzato (`OP_EQUALVERIFY`)
4. Verifica che la firma sia valida per la transazione corrente utilizzando la chiave pubblica (`OP_CHECKSIG`)

La transazione è autorizzata solo se l'esecuzione completa lascia un valore `TRUE` in cima allo stack, indicando che tutte le condizioni sono state soddisfatte, ovvero che il proponente ha dimostrato di possedere la chiave privata corrispondente alla chiave pubblica il cui hash era specificato nell'output.

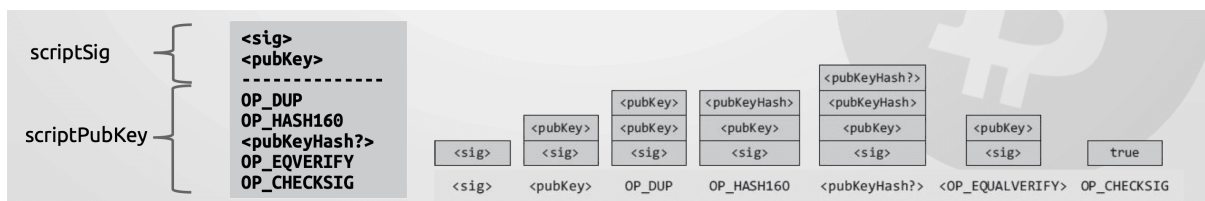


Figure 1.3: Processo di esecuzione di uno script Bitcoin P2PKH, con visualizzazione dello stato dello stack durante le fasi successive dell'esecuzione e del concatenamento tra scriptSig e scriptPubKey.

1.3.3 Applicazioni Avanzate di Bitcoin Script

Oltre al meccanismo P2PKH standard, Bitcoin Script supporta una gamma di applicazioni avanzate che estendono le funzionalità del sistema:

- **Multi-firma con threshold:** Script che richiedono più firme per autorizzare una transazione, implementando meccanismi di controllo distribuito (es. "2-di-3").
- **Timelock:** Congelamento temporaneo di fondi attraverso `OP_CHECKLOCKTIMEVERIFY`, abilitando contratti condizionati dal tempo.
- **Pay-to-Script-Hash (P2SH):** Meccanismo che permette di nascondere la complessità degli script, rendendo possibili schemi di pagamento avanzati con indirizzi di formato standard.
- **Proof-of-Burn:** Utilizzo di `OP_RETURN` per rendere deliberatamente non spendibili dei bitcoin, con possibili applicazioni in sistemi di timestamping e ancoraggio di dati.
- **Microtransazioni:** Implementazione di canali di pagamento per transazioni frequenti e di piccolo importo, fondamentali per soluzioni come Lightning Network.

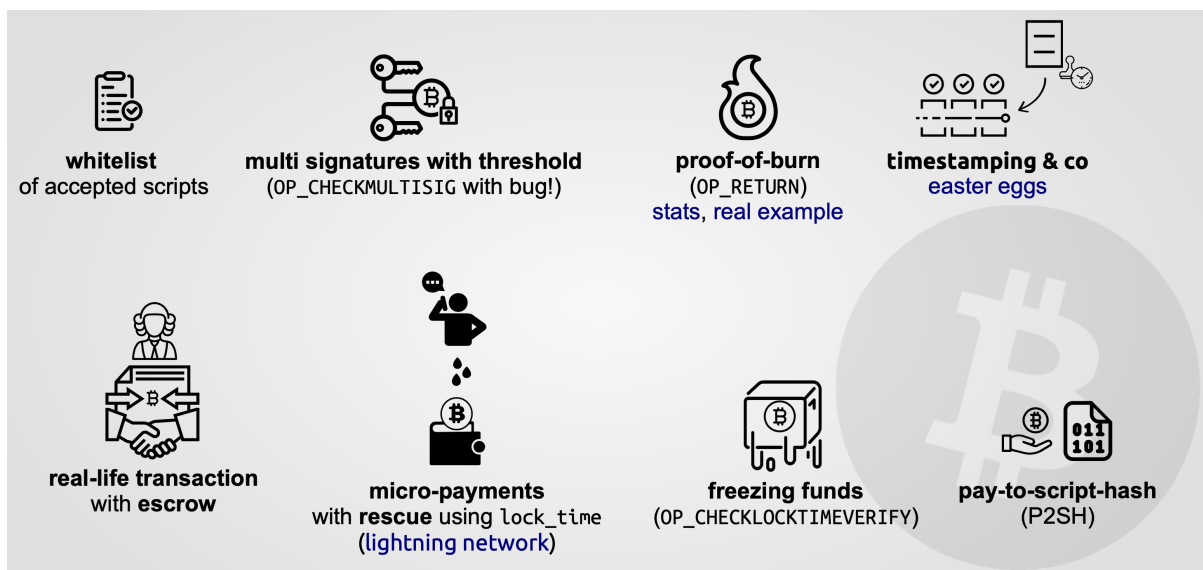


Figure 1.4: Panoramica delle applicazioni avanzate di Bitcoin Script, includendo multi-signature, proof-of-burn, timelock, pay-to-script-hash e micro-pagamenti.

La flessibilità di Bitcoin Script, benché limitata rispetto a linguaggi Turing-completi, fornisce un substrato adeguato per l'implementazione di contratti finanziari complessi, preservando al contempo la verificabilità e la sicurezza del sistema.

Chapter 2

Lezione 2

2.1 Struttura della Blockchain: Blocchi e Loro Organizzazione

La blockchain di Bitcoin è una struttura dati distribuita che organizza le transazioni in blocchi interconnessi, creando un registro immutabile e verificabile. Ogni blocco contiene un insieme di transazioni validate e collegamenti crittografici che garantiscono l'integrità dell'intera catena.

2.1.1 Struttura del Blocco

Un blocco Bitcoin è composto da due sezioni principali:

- **Header del blocco:** Contiene metadati e parametri di consenso, tra cui:
 - **hash:** Identificativo univoco del blocco.
 - **prev_block:** Hash del blocco precedente, creando un collegamento crittografico con la catena esistente.
 - **mrkl_root:** Radice dell'albero di Merkle delle transazioni contenute nel blocco.
 - **time:** Timestamp di creazione del blocco.
 - **bits:** Rappresentazione compatta del target di difficoltà corrente.
 - **nonce:** Valore variabile utilizzato nel processo di mining per risolvere il puzzle crittografico.
- **Corpo del blocco:** Contiene l'elenco completo delle transazioni e la struttura dell'albero di Merkle:
 - **n_tx:** Numero di transazioni incluse nel blocco.
 - **tx:** Array di transazioni complete.
 - **mrkl_tree:** Struttura dell'albero di Merkle utilizzato per verifiche efficienti.

solamente di un numero logaritmico di hash (rispetto al numero totale di transazioni nel blocco) per costruire una prova di appartenenza, rendendo possibile l'implementazione di client "leggeri" che non memorizzano l'intera blockchain.

2.1.3 Catena di Blocchi e Immutabilità

I blocchi sono organizzati in una struttura sequenziale lineare, dove ogni blocco contiene un riferimento crittografico (hash) al blocco precedente. Questa proprietà strutturale crea un'interdipendenza che garantisce l'immutabilità del registro:

- Qualsiasi modifica al contenuto di un blocco altererebbe il suo hash.
- L'alterazione dell'hash invaliderebbe tutti i blocchi successivi, che referenziano quel hash.
- Per modificare un blocco nel passato, un attaccante dovrebbe ricalcolare tutti i blocchi successivi con sufficiente potenza computazionale.

La sicurezza della blockchain deriva da questa proprietà strutturale: più un blocco è profondamente incorporato nella catena (coperto da molti blocchi successivi), più diventa computazionalmente proibitivo alterarlo. Questo meccanismo garantisce che, una volta che un blocco è stato confermato da un numero sufficiente di blocchi successivi, le transazioni in esso contenute possono essere considerate irreversibili con un'altissima probabilità.

Questa struttura a catena, combinata con il meccanismo di consenso Proof-of-Work, rende la blockchain progressivamente più resistente alle modifiche con l'aggiunta di nuovi blocchi, fornendo garanzie di immutabilità praticamente assolute per blocchi sufficientemente "profondi" nella catena.

2.2 Transazioni Speciali: Coinbase

2.2.1 Creazione di Nuovi Bitcoin

Ogni blocco contiene una transazione speciale denominata "coinbase", che svolge una funzione fondamentale nell'ecosistema Bitcoin:

- Rappresenta l'unico meccanismo attraverso cui nuovi bitcoin vengono introdotti in circolazione.
- Costituisce la principale ricompensa per i miners che dedicano risorse computazionali alla sicurezza della rete.
- Segue una struttura distintiva che la differenzia dalle transazioni ordinarie.

La transazione coinbase presenta caratteristiche strutturali peculiari:

- **Input speciale:** Utilizza un hash fittizio composto interamente da zeri e un valore di indice pari a 0xFFFFFFFF, indicando che non consuma UTXO preesistenti.
- **Campo coinbase:** Uno spazio libero che può contenere dati arbitrari (fino a 100 byte), storicamente utilizzato per messaggi o identificativi del mining pool.

- **Output:** Tipicamente contiene un singolo output diretto all'indirizzo del miner, con un valore corrispondente alla ricompensa di blocco più le commissioni delle transazioni incluse.

- Each block contains a **coinbase transaction**:
 - the only way to **create** new bitcoins;
 - **main remuneration** for the miners;
 - 50 BTC → ... → 6,25 BTC (May 2020) → 3,125 BTC (Aprile 2024)
 - a free field **coinbase**
 - **block 0**: "The Times 03/Jan/2009 Chancellor on brink of second bailout for banks"



remuneration

```

      "in": [
        {
          "prev_out": {
            "hash": "000000....0000000",
            "n": 4294967295 ( == 0xffffffff )
          },
          "coinbase": "... "
        }
      ],
      "out": [
        {
          "value": "6.351417", ( == 6.25 + 0.011417 )
          "scriptPubKey": "OPDUP OPHASH160 ... "
        }
      ]
    
```

Figure 2.2: Struttura di una transazione coinbase, evidenziando l'input speciale con hash di zeri, il campo coinbase personalizzabile e l'output che combina la ricompensa di blocco con le commissioni delle transazioni.

2.2.2 Schema di Ricompensa e Halving

Il sistema di ricompensa di Bitcoin segue un programma predefinito di progressiva riduzione dell'emissione:

- La ricompensa iniziale per blocco era di 50 BTC quando Bitcoin fu lanciato nel 2009.
- Ogni 210.000 blocchi (approssimativamente ogni 4 anni), la ricompensa viene dimezzata (halving).
- La progressione è stata: 50 BTC → 25 BTC (2012) → 12,5 BTC (2016) → 6,25 BTC (2020) → 3,125 BTC (2024).
- Questo schema continuerà fino all'esaurimento dell'emissione, con un limite massimo di 21 milioni di bitcoin.

Questo modello di emissione decrescente implementa una politica monetaria deflazionistica programmata matematicamente, senza necessità di interventi discrezionali. La prevedibilità dell'offerta costituisce una delle caratteristiche fondamentali di Bitcoin, contrapponendosi alla politica monetaria delle valute fiat, controllata dalle banche centrali.

Le implicazioni economiche di questo schema sono significative:

- Inflazione controllata e progressivamente decrescente.
- Prevedibilità dell'offerta a lungo termine.
- Aumento della scarsità relativa nel tempo.

In una prospettiva di lungo periodo, la diminuzione graduale della ricompensa di blocco comporterà un cambiamento nel modello di sostenibilità economica del mining, con una transizione da un sistema prevalentemente basato sulla ricompensa di nuova emissione a uno in cui le commissioni di transazione costituiranno l'incentivo principale per i miners.

2.2.3 Aspetti Storici e Culturali

La transazione coinbase del blocco genesis (blocco 0) riveste un'importanza storica e simbolica particolare, contenendo il messaggio:

"The Times 03/Jan/2009 Chancellor on brink of second bailout for banks"

Questo riferimento all'articolo del Times riguardante il salvataggio bancario durante la crisi finanziaria del 2008 è interpretato come un commento critico sul sistema finanziario tradizionale e una dichiarazione di intenti per Bitcoin come alternativa decentralizzata.

Il messaggio inciso nel blocco genesis costituisce un manifesto implicito dell'ethos di Bitcoin, evidenziando la motivazione filosofica e politica che ha ispirato la creazione della criptovaluta: fornire un'alternativa al sistema finanziario centralizzato che aveva dimostrato le proprie fragilità durante la crisi finanziaria globale.

Successivamente, il campo coinbase è stato utilizzato per vari scopi, tra cui:

- Identificazione dei mining pool e software utilizzati.
- Messaggi personali o politici.
- Dati tecnici utili per l'ottimizzazione del mining.

Questa possibilità di inserire dati arbitrari nel campo coinbase ha contribuito a sviluppare ulteriori applicazioni della blockchain Bitcoin come strumento di timestamping e notarizzazione, aprendo la strada a utilizzi che vanno oltre la semplice transazione di valore.

Chapter 3

Lezione 3

3.1 Rete Bitcoin: Architettura di Rete

La rete Bitcoin è implementata come un sistema peer-to-peer (P2P) completamente decentralizzato, dove tutti i nodi svolgono funzioni equivalenti senza gerarchie predefinite. Questa architettura contribuisce alla resilienza e alla resistenza alla censura del sistema.

3.1.1 Caratteristiche della Rete P2P

La rete Bitcoin possiede diverse proprietà distintive:

- **Egalitarismo strutturale:** Tutti i nodi operano secondo le stesse regole e protocolli, senza nodi privilegiati o centralizzati.
- **Connessioni TCP:** Le comunicazioni avvengono attraverso connessioni TCP standard, tipicamente sulla porta 8333.
- **Topologia logica randomizzata:** La struttura di connessione non riflette la disposizione fisica dei nodi, ma si forma attraverso meccanismi probabilistici di discovery.
- **Dinamicità:** Nodi possono unirsi o abbandonare la rete in qualsiasi momento senza compromettere il funzionamento complessivo del sistema.

L'architettura paritaria della rete Bitcoin elimina i punti singoli di fallimento tipici dei sistemi centralizzati. Ogni nodo è potenzialmente connesso a numerosi altri nodi, creando una rete mesh robusta dove l'informazione può propagarsi attraverso percorsi multipli. Questa ridondanza strutturale garantisce che il sistema rimanga operativo anche in caso di perdita di connettività di una porzione significativa dei nodi.

La topologia di rete risultante è una struttura mesh parzialmente connessa, dove ogni nodo mantiene connessioni con un sottoinsieme limitato di altri nodi (tipicamente 8-125 connessioni attive).

3.1.2 Meccanismi di Discovery

I nuovi nodi che si uniscono alla rete necessitano di un meccanismo per scoprire i peer esistenti. Bitcoin implementa diverse strategie complementari:

- **DNS seeds:** Server DNS specializzati che forniscono indirizzi IP di nodi attivi conosciuti.
- **Nodi hardcoded:** Alcuni indirizzi di nodi storicamente stabili sono inclusi nel codice sorgente come punto di partenza.
- **Protocollo getaddr:** Una volta stabilita una connessione iniziale, un nodo può richiedere liste di altri nodi conosciuti attraverso questo comando.
- **Gossip di indirizzi:** I nodi condividono periodicamente e spontaneamente indirizzi di altri peer, facilitando la propagazione delle informazioni di rete.

Il processo di bootstrap di un nuovo nodo inizia tipicamente con la query a DNS seeds o la connessione a nodi hardcoded. Una volta stabilita almeno una connessione, il nodo invia richieste `getaddr()` per ottenere informazioni su altri potenziali peer. Questo meccanismo a cascata permette ai nuovi nodi di costruire rapidamente un set di connessioni adeguato.

Questi meccanismi garantiscono che, anche in caso di significativa fluttuazione nella partecipazione, la rete mantenga un grado adeguato di connettività e resilienza. La natura distribuita della discovery dei peer rende il sistema resistente a tentativi di censura o isolamento di parti della rete.

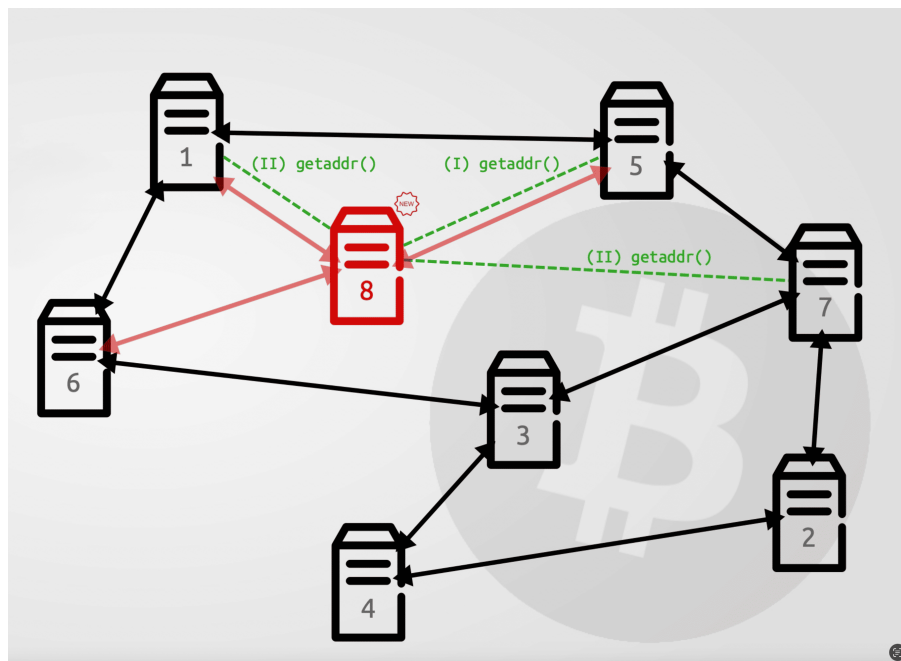


Figure 3.1: Rappresentazione schematica della rete Bitcoin P2P, con visualizzazione delle connessioni tra nodi e del processo di discovery attraverso il protocollo `getaddr`.

3.2 Propagazione di Transazioni

La diffusione delle transazioni attraverso la rete Bitcoin avviene secondo un protocollo di gossip (flooding), progettato per garantire una propagazione rapida ed efficiente senza necessità di coordinamento centralizzato.

3.2.1 Protocollo di Gossip

Quando un nodo riceve una nuova transazione, il processo di propagazione segue questi passaggi:

1. Il nodo verifica la validità della transazione secondo le regole di consenso.
2. Se la transazione è valida e non era già conosciuta, viene inserita nel transaction pool locale.
3. Il nodo trasmette la transazione a tutti i peer con cui mantiene una connessione attiva.
4. I nodi riceventi ripetono il processo, creando un effetto di diffusione esponenziale.

Questo meccanismo garantisce che le transazioni valide raggiungano rapidamente la maggior parte della rete, tipicamente in pochi secondi. La propagazione esponenziale assicura che anche in reti con diametro significativo (numero massimo di hop tra due nodi qualsiasi), l'informazione si diffonda con una latenza contenuta.

Il protocollo di gossip implementa un equilibrio efficace tra completezza e efficienza: ogni transazione valida viene potenzialmente inoltrata a tutti i nodi della rete, ma senza ridondanza superflua grazie al tracciamento delle transazioni già conosciute.

3.2.2 Transaction Pool

Ogni nodo mantiene un pool di transazioni valide ma non ancora confermate in un blocco, denominato mempool. La gestione di questa struttura dati comporta diverse considerazioni:

- **Controlli di validità:** Prima dell'inclusione nel pool, ogni transazione viene sottoposta a verifiche che includono:
 - Validità sintattica e semantica della struttura dati.
 - Conformità agli script whitelist riconosciuti dal nodo.
 - Assenza di duplicazione nel pool esistente.
 - Compatibilità con altre transazioni nel pool (assenza di conflitti di spesa).
- **Condizioni di race:** Transazioni in conflitto (che tentano di spendere lo stesso UTXO) generano una situazione competitiva dove tipicamente "vince il primo arrivato", sebbene esistano meccanismi come Replace-by-Fee (RBF) che permettono la sostituzione condizionata.
- **Sincronizzazione imperfetta:** I pool di transazioni dei diversi nodi non sono perfettamente sincronizzati, creando una visione temporaneamente eterogenea dello stato della rete prima della conferma definitiva nei blocchi.

Questa gestione distribuita delle transazioni in attesa costituisce un esempio di consenso emergente, dove la rete converge progressivamente verso una visione condivisa senza necessità di coordinamento esplicito.

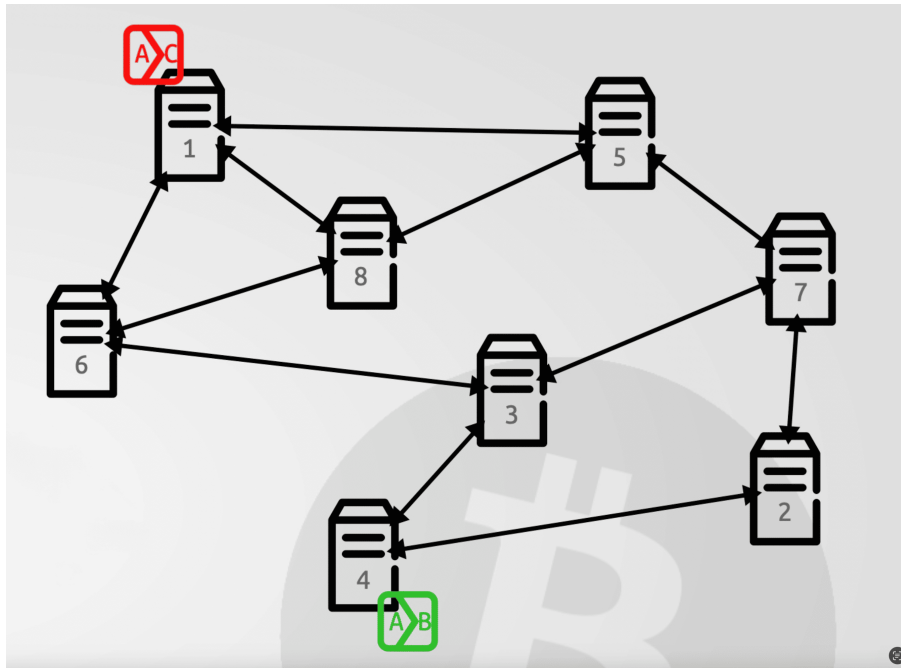


Figure 3.2: Processo di propagazione di una nuova transazione attraverso la rete Bitcoin, con evidenziazione delle verifiche effettuate da ciascun nodo e della dinamica di diffusione tramite il protocollo di gossip.

3.3 Propagazione dei Blocchi

La diffusione dei nuovi blocchi segue un processo analogo a quello delle transazioni, ma con verifiche più stringenti e implicazioni più significative per il consenso distributivo.

3.3.1 Criteri di Validazione e Propagazione

Un nuovo blocco viene propagato attraverso la rete solo se soddisfa condizioni rigorose:

- **Soluzione del puzzle crittografico:** L'hash del blocco deve essere inferiore al target di difficoltà corrente, dimostrando il completamento del lavoro di Proof-of-Work.
- **Validità delle transazioni:** Tutte le transazioni incluse nel blocco devono rispettare le regole di consenso della rete.
- **Estensione della catena principale:** Il blocco deve costruire sulla catena più lunga conosciuta, rispettando la regola del "longest chain".

Quando un miner scopre un nuovo blocco valido, lo trasmette immediatamente ai suoi peer. Ogni nodo che lo riceve verifica la sua validità e, se soddisfa tutti i requisiti, lo inoltra ai propri peer e lo aggiunge alla propria copia locale della blockchain. Questo meccanismo di propagazione a cascata garantisce che i blocchi validi si diffondano rapidamente attraverso l'intera rete.

Questi controlli assicurano che solo blocchi legittimi vengano incorporati nella blockchain, prevenendo attacchi e divergenze non intenzionali.

3.3.2 Statistiche di Propagazione

La velocità di propagazione dei blocchi rappresenta un parametro critico per la sicurezza e l'efficienza della rete:

- Il tempo di propagazione dipende principalmente dalla dimensione del blocco e dalla connettività della rete.
- Statistiche empiriche (2023) indicano che un blocco raggiunge:
 - Circa il 50% dei nodi in 3-4 secondi.
 - Circa il 90% dei nodi in 9-14 secondi.
- Questi tempi sono influenzati da fattori come la distribuzione geografica dei nodi e la variabilità della larghezza di banda disponibile.

La minimizzazione dei tempi di propagazione rappresenta un obiettivo importante per l'ottimizzazione del protocollo, in quanto ritardi nella diffusione dei blocchi possono aumentare la probabilità di fork temporanei e ridurre l'efficienza complessiva del mining.

3.4 Tipologie di Nodi

La rete Bitcoin comprende diverse categorie di nodi con caratteristiche, requisiti e funzionalità differenziate, permettendo vari livelli di partecipazione in base alle risorse disponibili.

3.4.1 Full Nodes

I nodi completi rappresentano la spina dorsale della rete Bitcoin, garantendo la verifica indipendente e la propagazione delle transazioni e dei blocchi:

- **Connettività permanente:** Mantengono una presenza costante nella rete per massimizzare la resilienza del sistema.
- **Archiviazione completa:** Conservano l'intera blockchain, comprensiva di tutte le transazioni dalla genesi (460 GB nel 2023).
- **Verifica esaustiva:** Validano indipendentemente ogni transazione e blocco secondo tutte le regole di consenso.
- **Ottimizzazione UTXO:** Mantengono una cache del set UTXO in memoria RAM per accelerare le verifiche delle transazioni.

Questi nodi forniscono il livello massimo di sicurezza e indipendenza, in quanto non necessitano di fidarsi di terze parti per la validazione delle transazioni che li riguardano. Rappresentano la manifestazione concreta del principio "don't trust, verify" fondamentale nella filosofia Bitcoin.

La distribuzione geografica e numerica dei full node ha implicazioni significative per la resilienza della rete. Una rete con molti nodi completi distribuiti globalmente offre maggiore resistenza a censura, interruzioni localizzate e tentativi di manipolazione.

3.4.2 Nodi Lightweight (SPV)

I nodi Simplified Payment Verification (SPV) offrono un'alternativa più leggera per dispositivi con risorse limitate, come smartphone o computer con spazio di archiviazione ridotto:

- **Archiviazione selettiva:** Memorizzano solo gli header dei blocchi (circa 460 MB vs. 460 GB), riducendo drasticamente i requisiti di storage.
- **Verifica mirata:** Verificano solo le transazioni di diretto interesse per l'utente, non l'intera blockchain.
- **Dipendenza dai full node:** Richiedono la connessione a full node per ottenere dati sulle transazioni specifiche e le relative prove di inclusione.
- **Verifica basata su Merkle proof:** Utilizzano prove crittografiche compatte basate sull'albero di Merkle per confermare l'inclusione di transazioni in blocchi validi.

I nodi SPV offrono un equilibrio tra efficienza e sicurezza, permettendo la partecipazione alla rete con risorse limitate, ma accettando un modello di fiducia delegata verso i full node per alcune verifiche.

3.4.3 Distribuzione Geografica e Statistica

La distribuzione dei nodi Bitcoin fornisce indicazioni sulla decentralizzazione e resilienza della rete:

- Nel 2023, la rete contava approssimativamente 16.000 nodi pubblicamente accessibili.
- La distribuzione percentuale per tipologia di nodo mostrava:
 - IPv4: circa 60% dei nodi
 - IPv6: circa 13% dei nodi
 - Onion (Tor): circa 27% dei nodi
- La distribuzione geografica evidenzia concentrazioni significative in Nord America, Europa e alcune parti dell'Asia, riflettendo fattori come disponibilità di banda larga, regolamentazione favorevole e adozione tecnologica.

Questa distribuzione, benché imperfetta, contribuisce alla resistenza della rete contro censura geograficamente localizzata e interruzioni di servizio regionali. La presenza di un numero significativo di nodi che operano attraverso Tor rappresenta un ulteriore livello di protezione contro tentativi di censura basati su indirizzo IP.

Chapter 4

Lezione 4

4.1 Limitazioni e Scalabilità di Bitcoin: Vincoli di Progettazione

Bitcoin è stato concepito con parametri e limiti specifici che ne determinano le caratteristiche operative fondamentali. Questi vincoli, incorporati nel protocollo originale, definiscono l'identità tecnico-economica del sistema e le sue potenzialità di evoluzione.

4.1.1 Parametri Fondamentali

I principali vincoli strutturali di Bitcoin includono:

- **Intervallo tra blocchi:** Calibrato per generare un nuovo blocco approssimativamente ogni 10 minuti attraverso l'aggiustamento dinamico della difficoltà di mining.
- **Dimensione dei blocchi:** Limitata storicamente a circa 1 MB, sebbene modifiche come SegWit abbiano incrementato la capacità effettiva.
- **Limite di operazioni crittografiche:** Un massimo di circa 20.000 verifiche di firma per blocco, determinando un limite pratico al numero di transazioni processabili.
- **Offerta monetaria massima:** Rigidamente fissata a 21 milioni di bitcoin, con una progressione di emissione predefinita.
- **Schema di ricompensa:** Sequenza predeterminata di halving che dimezza la ricompensa del blocco approssimativamente ogni quattro anni.

La combinazione di questi parametri ha conseguenze significative sulle capacità operative della rete. In particolare, la dimensione limitata dei blocchi e l'intervallo fisso tra di essi generano un throughput massimo teorico di circa 7 transazioni al secondo. Questo valore è significativamente inferiore rispetto ai sistemi di pagamento tradizionali, come Visa o Mastercard, che possono gestire migliaia di operazioni nello stesso intervallo.

Le limitazioni di scalabilità rappresentano uno dei principali punti di dibattito nella comunità Bitcoin. Da un lato, i vincoli di throughput limitano l'adozione di massa per transazioni quotidiane; dall'altro, i sostenitori argomentano che questi limiti sono

necessari per mantenere la decentralizzazione della rete, permettendo la partecipazione di nodi con risorse computazionali e di banda limitate.

4.1.2 Primitive Crittografiche

Bitcoin utilizza un set specifico e immutabile di algoritmi crittografici come fondamento del suo funzionamento:

- **Firma digitale:** ECDSA (Elliptic Curve Digital Signature Algorithm) sulla curva secp256k1.
- **Funzione hash:** SHA-256, utilizzata sia per l'identificazione delle transazioni e dei blocchi che per il puzzle di Proof-of-Work.

La scelta di queste primitive riflette considerazioni di sicurezza, efficienza e disponibilità al momento della progettazione. Tuttavia, la difficoltà di modificarle senza hard fork impone rigidità nell'adattamento a progressi crittografici successivi.

L'evoluzione delle tecnologie quantistiche, in particolare, rappresenta una sfida potenziale per la sicurezza a lungo termine di queste primitive. Gli algoritmi a curva ellittica come ECDSA sono vulnerabili all'algoritmo di Shor, che potrebbe essere implementato su computer quantistici sufficientemente potenti. Questo scenario ha stimolato discussioni sulla necessità di pianificare transizioni future verso primitive crittografiche resistenti agli attacchi quantistici.

4.2 Meccanismi di Evoluzione del Protocollo

Nonostante i vincoli strutturali, Bitcoin dispone di meccanismi che permettono l'evoluzione del protocollo, bilanciando l'esigenza di stabilità con la necessità di adattamento e miglioramento.

4.2.1 Hard Fork vs. Soft Fork

Le modifiche al protocollo Bitcoin possono essere implementate attraverso due approcci principali:

- **Hard Fork:** Rappresenta un cambiamento non retrocompatibile che richiede l'aggiornamento di tutti i nodi. Caratteristiche:
 - I blocchi conformi alle nuove regole vengono rifiutati dai nodi non aggiornati.
 - Crea una biforcazione permanente nella blockchain, con nodi e miners potenzialmente divisi su catene divergenti.
 - Comporta rischi di frammentazione della rete e dell'ecosistema fino al completamento della transizione.
- **Soft Fork:** Implementa modifiche retrocompatibili che restringono o precisano le regole esistenti. Caratteristiche:
 - I blocchi conformi alle nuove regole risultano validi anche per i nodi non aggiornati.

- Richiede solamente che una maggioranza di miners adotti le nuove regole per garantirne l'applicazione.
- Presenta rischi minori ma ancora significativi, come la possibilità che nodi non aggiornati creino blocchi invalidi secondo le nuove regole.

La comunità Bitcoin ha storicamente privilegiato soft fork per minimizzare i rischi di frammentazione, utilizzando tecniche di progettazione che permettono significative innovazioni mantenendo la retrocompatibilità.

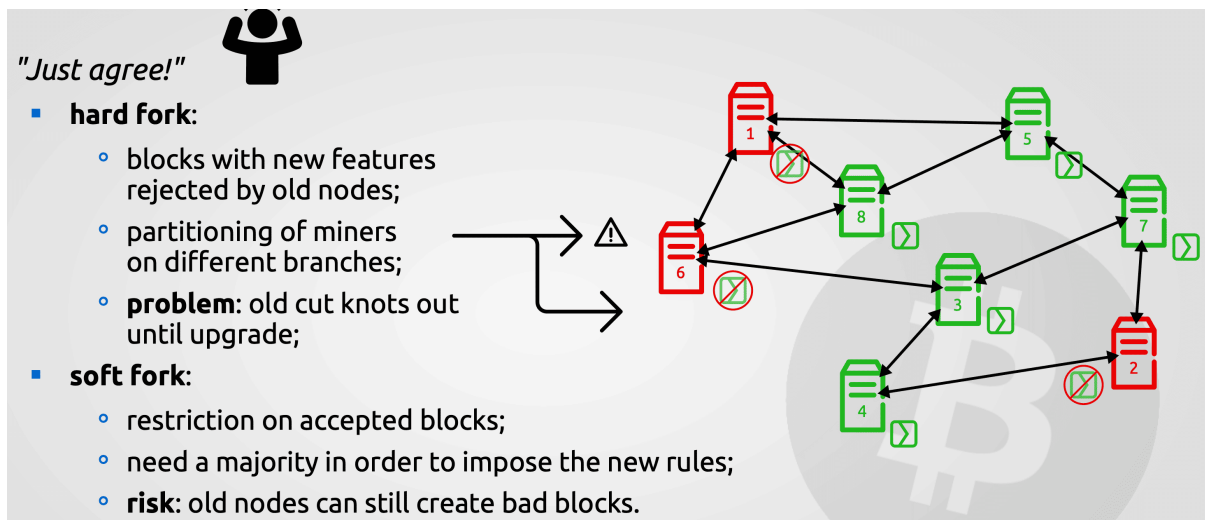


Figure 4.1: Rappresentazione schematica della differenza tra hard fork e soft fork, con evidenza degli effetti sulla compatibilità tra nodi aggiornati e non aggiornati.

4.3 Fondamenti della Proprietà Crittografica

In Bitcoin, la proprietà di fondi è definita esclusivamente attraverso il controllo crittografico: chi possiede la chiave privata corrispondente a un determinato indirizzo può disporre dei bitcoin associati a quell'indirizzo. Questa caratteristica trasforma la gestione delle chiavi private in un aspetto cruciale della sicurezza.

4.3.1 Relazione tra Chiavi e Identità

La proprietà in Bitcoin è implementata attraverso un sistema di crittografia asimmetrica:

- **Chiave privata:** Sequenza numerica segreta che permette di generare firme digitali valide per autorizzare transazioni. Rappresenta di fatto il pieno controllo sui fondi.
- **Chiave pubblica:** Derivata matematicamente dalla chiave privata attraverso operazioni sulla curva ellittica secp256k1.
- **Indirizzo Bitcoin:** Hash della chiave pubblica, utilizzato come "destinatario" nelle transazioni per ragioni di compattezza e sicurezza aggiuntiva.

L'equazione fondamentale della proprietà in Bitcoin può essere sintetizzata come: possesso dei bitcoin = possesso della chiave privata. Questa equivalenza comporta implicazioni significative: il furto o la perdita della chiave privata corrisponde inevitabilmente al furto o alla perdita permanente dei bitcoin associati.

Questa concezione di proprietà basata sul controllo crittografico differisce radicalmente dai sistemi finanziari tradizionali, dove la proprietà è tipicamente attestata da registri centralizzati e garantita da istituzioni fiduciarie. In Bitcoin, la proprietà è auto-verificabile e non richiede l'intermediazione di terzi per essere certificata o trasferita.

4.4 Wallet: Funzionalità e Tipologie

I wallet Bitcoin sono software o dispositivi che gestiscono le chiavi private e facilitano le operazioni di creazione e firma delle transazioni. La loro funzione primaria non è "contenere" bitcoin (che esistono solo come registrazioni sulla blockchain) ma gestire le chiavi necessarie per controllarli.

4.4.1 Caratteristiche Fondamentali

Un wallet Bitcoin completo implementa diverse funzionalità essenziali:

- **Generazione di indirizzi:** Creazione di nuovi indirizzi Bitcoin per la ricezione di fondi, presentati in formato base58 o come codici QR per facilitare il trasferimento.
- **Gestione delle chiavi private:** Archiviazione sicura delle chiavi private corrispondenti agli indirizzi generati.
- **Firma di transazioni:** Utilizzo delle chiavi private per autorizzare il trasferimento di fondi verso altri indirizzi.
- **Monitoraggio degli UTXO:** Tracciamento dei bitcoin disponibili attraverso l'interrogazione della blockchain.
- **Calcolo delle commissioni:** Stima e suggerimento di commissioni appropriate in base alla congestione della rete.

4.4.2 Indirizzi e Pseudonimia

Gli indirizzi Bitcoin svolgono un ruolo cruciale nella gestione della privacy e della sicurezza:

- **Formato standard:** Gli indirizzi Bitcoin utilizzano codifica base58, con inclusione di checksum per prevenire errori di trascrizione (es. "1A1zP1eP5QGefi2DMPTfTL5SLmv7DivfNa").
- **Indirizzi vanity:** Indirizzi personalizzati che iniziano con caratteri specifici scelti dall'utente (es. "1SatoshiHHqnDPRsfZ5GxJ8Gk9dbjO"), generati attraverso ricerca esaustiva.
- **Uso multiplo:** La creazione e l'utilizzo di molteplici indirizzi per diverse transazioni rafforza la pseudonimia, rendendo più complessa l'analisi delle connessioni tra transazioni.

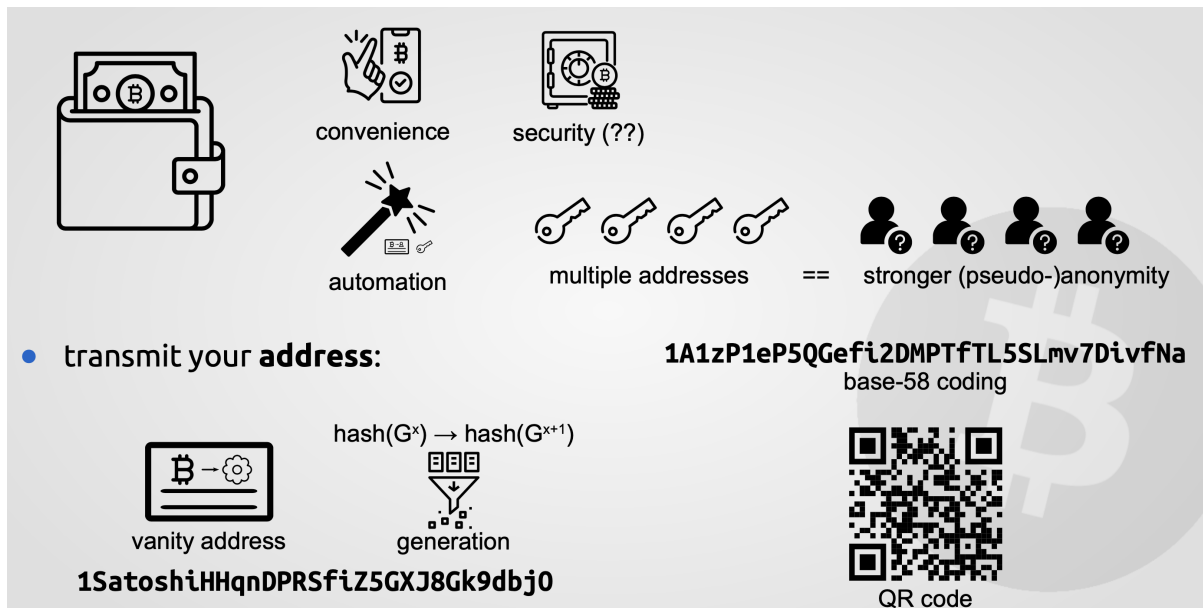


Figure 4.2: Funzionalità principali di un wallet Bitcoin, con esempi di indirizzi in formato base58 e codice QR, evidenziando il concetto di pseudonimia attraverso l'uso di indirizzi multipli.

Best practice di sicurezza raccomandano di utilizzare un nuovo indirizzo per ogni transazione in entrata, minimizzando il rischio di correlazione e aumentando la privacy.

4.5 Strategia di Storage: Hot vs. Cold

La gestione delle chiavi private comporta un trade-off fondamentale tra accessibilità e sicurezza, concretizzato nella distinzione tra storage "caldo" e "freddo".

4.5.1 Hot Storage

Il hot storage mantiene le chiavi private in ambienti connessi alla rete, privilegiando la convenienza operativa:

- **Wallet online:** Servizi web che gestiscono le chiavi per conto dell'utente, offrendo accessibilità da qualsiasi dispositivo.
- **Wallet desktop:** Applicazioni installate su computer personali con vari livelli di protezione.
- **Wallet mobile:** Applicazioni per smartphone che permettono pagamenti in mobilità e scansione di codici QR.

Il hot storage è indicato principalmente per importi modesti utilizzati frequentemente, analogamente al contante portato nel portafoglio fisico, a causa della maggiore esposizione a rischi di sicurezza informatica.

4.5.2 Cold Storage

Il cold storage mantiene le chiavi private completamente disconnesse dalla rete, massimizzando la protezione contro attacchi informatici:

- **Hardware wallet:** Dispositivi specializzati con microcontrollori sicuri che generano e archiviano chiavi private senza mai esporle al sistema operativo del computer host.
- **Paper wallet:** Trascrizione fisica delle chiavi private su supporto cartaceo, spesso accompagnata da codici QR.
- **Storage su dispositivi offline:** Archiviazione delle chiavi su dispositivi non connessi come USB drive o computer air-gapped.

Il cold storage è raccomandato per la custodia di importi significativi destinati a conservazione a lungo termine, analogamente a un conto di risparmio o cassetta di sicurezza.

4.5.3 Flussi Operativi Ibridi

Per combinare sicurezza e usabilità, molti utenti implementano strategie ibride:

- **Segregazione per valore:** Utilizzo di hot wallet per importi operativi e cold storage per riserve principali.
- **Firma offline:** Preparazione delle transazioni su dispositivi online, firma su dispositivi offline, e trasmissione finale da dispositivi online.
- **Indirizzi di osservazione:** Monitoraggio di saldi associati a chiavi in cold storage attraverso wallet configurati con le sole chiavi pubbliche.

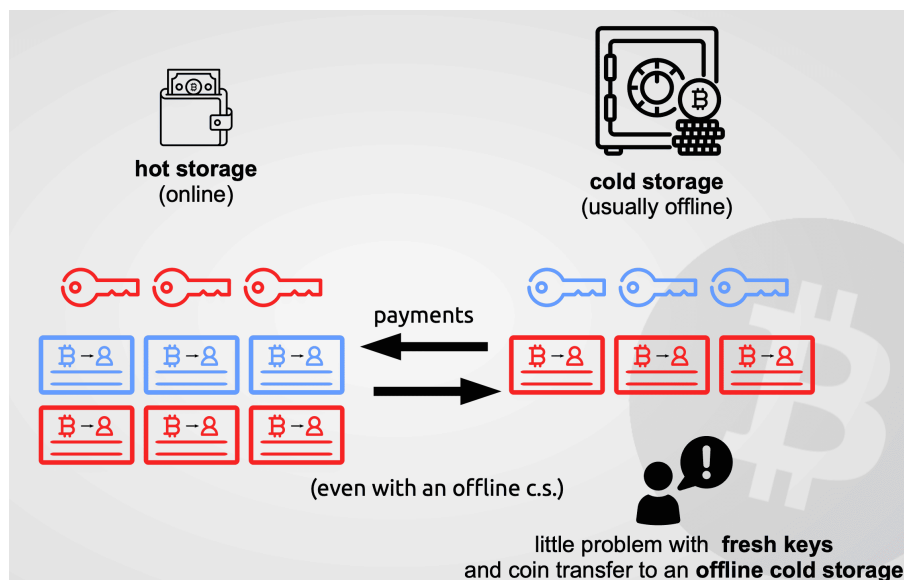


Figure 4.3: Confronto tra hot storage e cold storage, con visualizzazione dei flussi di pagamento e delle strategie di protezione delle chiavi in un sistema ibrido che combina sicurezza e convenienza operativa.

Questi approcci permettono di mantenere la maggior parte dei fondi al sicuro, minimizzando al contempo l'attrito operativo per transazioni regolari.

4.6 Wallet Deterministici Gerarchici (HD Wallet)

I wallet deterministici gerarchici (HD wallet), specificati principalmente dallo standard BIP-32 (Bitcoin Improvement Proposal 32), rappresentano un'evoluzione cruciale nella gestione delle chiavi crittografiche, in particolare per criptovalute come Bitcoin. Essi affrontano efficacemente le problematiche legate al backup e alla privacy tramite un sistema strutturato di derivazione matematica delle chiavi. L'idea centrale è generare un intero albero di coppie di chiavi (pubblica e privata) a partire da un singolo dato segreto iniziale.

Il funzionamento di un HD wallet si basa sulla generazione e derivazione strutturata di chiavi:

- **Seed iniziale:** È il punto di partenza. Solitamente si tratta di una sequenza mnemonica di 12-24 parole, come definito dallo standard BIP-39. Queste parole vengono convertite in un valore binario (il seed vero e proprio) che è l'unica informazione che l'utente deve custodire gelosamente per recuperare l'intero wallet e tutte le chiavi associate.
- **Master Key:** Dal seed binario, attraverso una funzione di derivazione chiave unidirezionale (come HMAC-SHA512), si ottengono due elementi fondamentali:
 - La master key (privata), che denoteremo con y .
 - Un chain code, ovvero una sequenza di 256 bit, che indicheremo con k .

La coppia (y, k) costituisce la master key estesa, detta **xpriv**. Dalla chiave y si può derivare la master key pubblica come G^y (dove G è il punto generatore della curva ellittica). La master key pubblica estesa **xpub** è composta dalla coppia (G^y, k) .

Le chiavi private e pubbliche **non** estese possono rispettivamente firmare transazioni e verificare le firme fatte con la chiave privata corrispondente (oltre che generare un indirizzo tramite opportuna hash function). Il Problema delle chiavi non estese è che se si usa una nuova coppia chiave privata/pubblica (e quindi un nuovo indirizzo) per ogni transazione per migliorare la privacy, ci si ritrova con tantissime chiavi private da backuppare. Perderne anche solo una significa perdere i fondi associati. Il sistema a chiavi estese sopperisce proprio a questa problematica.

Le controparti private preservano queste funzionalità ma possono anche fare altro. In particolare, dalla chiave privata estesa xpriv è possibile generare un intero "ramo" di chiavi figlie (sia xpriv figlie che xpub figlie). Dalla chiave pubblica estesa xpub è possibile generare un intero "ramo" di chiavi pubbliche figlie (e quindi indirizzi) senza conoscere alcuna chiave privata.

- **Chain Code:** Come accennato, il chain code è una sequenza di 256 bit. Ad ogni passaggio di derivazione, insieme alla nuova chiave (privata o pubblica) viene generato un nuovo chain code. Questo assicura che ogni "ramo" dell'albero di chiavi sia crittograficamente distinto e, cosa molto importante, permette la derivazione di chiavi pubbliche figlie a partire da una chiave pubblica genitrice senza compromettere la sicurezza.
- **Derivazione Gerarchica:** A partire dalla master key estesa (xpriv o xpub), è possibile generare "rami" di chiavi figlie. Ogni chiave figlia (e il suo rispettivo chain code) è derivata deterministicamente dalla sua chiave genitrice, dal chain

code genitore e da un numero indice (un intero a 32 bit). Questo processo può essere ripetuto ricorsivamente per creare una struttura ad albero (gerarchia) di chiavi.

Questa struttura gerarchica consente di generare un numero virtualmente illimitato di indirizzi e chiavi da un singolo backup (il seed), semplificando notevolmente la gestione della sicurezza e migliorando la privacy (ad esempio, utilizzando un nuovo indirizzo per ogni transazione ricevuta).

4.6.1 Separazione delle Funzionalità e Meccanismo di Derivazione

Una delle caratteristiche più significative e utili degli HD wallet è la capacità di separare la funzione di generazione degli indirizzi dalla capacità di firmare transazioni (che richiede le chiavi private).

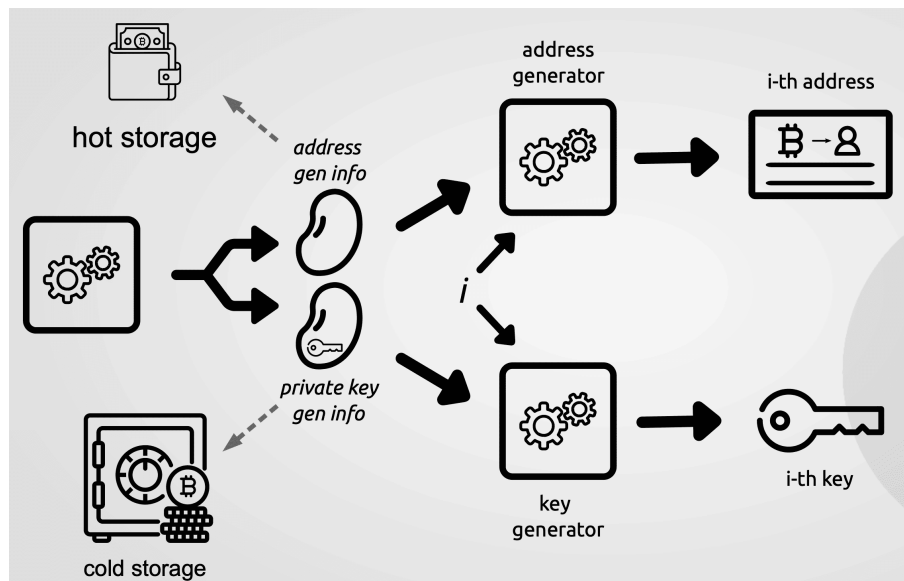


Figure 4.4: Struttura di un wallet deterministico gerarchico. Dal seed (non mostrato, ma implicito) si derivano le informazioni per la generazione di indirizzi (“address gen info”, xpub) e per la generazione di chiavi private (“private key gen info”, xpriv). Un indice i permette di derivare l’ i -esimo indirizzo e la corrispondente i -esima chiave privata. Le formule del caso ECDSA mostrano come il chain code k e l’indice i contribuiscono alla derivazione.

La derivazione dell’ i -esima chiave figlia avviene come segue:

- **Address Generation Info:** Queste sono contenute nella chiave pubblica estesa (xpub), ovvero la coppia (G^y, k) , con G^y chiave pubblica genitrice e k chain code genitore.

La chiave pubblica dell’ i -esimo indirizzo (G^{x_i}) si ottiene calcolando un valore a partire dal chain code k e dall’indice i , e combinandolo con la chiave pubblica genitrice y :

$$G^{x_i} = G^y \cdot G^{\text{hash}(k||i)}$$

L’indirizzo dell’ i -esimo utente è un hash della chiave pubblica G^{x_i} .

- **Private Key Gen Info:** Queste sono contenute nella chiave privata estesa (xpriv), ovvero la coppia (y, k) , con y chiave privata genitrice e k chain code genitore.

La chiave privata dell' i -esimo indirizzo (x_i) si ottiene combinando la chiave privata genitrice y con lo stesso valore derivato dal chain code k e dall'indice i usato per la chiave pubblica:

$$x_i = (y + \text{hash}(k \parallel i)) \mod n$$

dove n è l'ordine del gruppo.

È fondamentale notare che la componente $\text{hash}(k \parallel i)$ è la stessa in entrambe le derivazioni. Questo assicura che la chiave pubblica G^{x_i} generata dalla xpub sia effettivamente la chiave pubblica corrispondente alla chiave privata x_i generata dalla xpriv. La funzione hash qui rappresenta un processo crittografico deterministico che prende in input il chain code e l'indice.

4.7 Fondamenti di ECDSA

ECDSA rappresenta l'algoritmo di firma digitale utilizzato in Bitcoin per autenticare le transazioni, garantendo che solo il legittimo proprietario di una chiave privata possa autorizzare il movimento dei fondi corrispondenti.

4.7.1 Parametri e Componenti

L'implementazione ECDSA in Bitcoin utilizza specifici parametri crittografici:

- **Parametri pubblici condivisi:** Includono il punto generatore G della curva ellittica secp256k1 e l'ordine q del gruppo ciclico generato da G .
- **Chiave privata:** Un intero x scelto casualmente nell'intervallo $\{1, \dots, q - 1\}$.
- **Chiave pubblica:** Il punto $Y = G^x$ sulla curva ellittica, ottenuto attraverso moltiplicazione scalare del generatore per la chiave privata.

Questi elementi costituiscono l'infrastruttura crittografica su cui si basa l'intero sistema di autenticazione di Bitcoin.

4.7.2 Processo di Firma

La generazione di una firma ECDSA per un messaggio (transazione) comprende i seguenti passaggi:

1. Selezione di un numero casuale k nell'intervallo $\{1, \dots, q - 1\}$.
2. Calcolo del punto $R = G^k$ sulla curva ellittica.
3. Estrazione dell'ascissa r del punto R come primo componente della firma.
4. Calcolo dell'hash m del messaggio da firmare.
5. Calcolo del secondo componente della firma: $s = k^{-1}(m + r \cdot x) \mod q$.
6. Produzione della firma completa come coppia (r, s) .

La sicurezza di questo processo dipende criticamente dalla generazione del valore casuale k , che deve essere:

- Generato con sufficiente entropia (in maniera abbastanza imprevedibile).
- Utilizzato una sola volta (never reused).
- Mantenuto segreto durante e dopo il processo di firma.

Fallimenti storici nell'implementazione di queste protezioni hanno portato a compromissioni catastrofiche, come nei casi della PlayStation 3 e di alcune implementazioni Android, dove la riutilizzazione accidentale di valori k ha permesso l'estrazione delle chiavi private corrispondenti.

4.7.3 Processo di Verifica

La verifica di una firma ECDSA (r, s) per un messaggio richiede i seguenti passaggi:

1. Calcolo dell'hash m del messaggio originale.
2. Calcolo dei coefficienti $u_1 = m \cdot s^{-1} \bmod q$ e $u_2 = r \cdot s^{-1} \bmod q$.
3. Calcolo del punto $R = G^{u_1} \cdot Y^{u_2}$ attraverso operazioni sulla curva ellittica.
4. Verifica che l'ascissa del punto R sia congruente a r modulo q .

Questo processo può essere eseguito da qualsiasi entità in possesso della chiave pubblica del firmatario, rendendo la verifica una operazione completamente pubblica e decentralizzata.

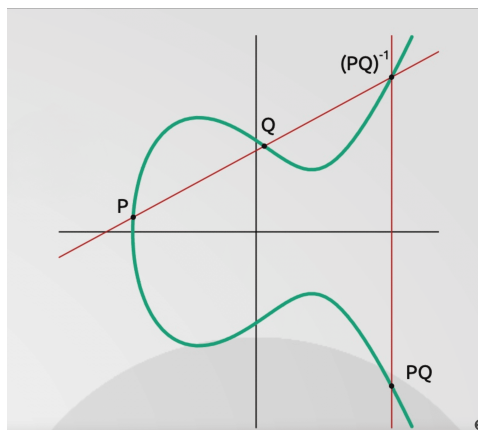


Figure 4.5: Rappresentazione grafica della curva ellittica secp256k1 utilizzata in Bitcoin, con visualizzazione delle operazioni di punto che sottendono gli algoritmi ECDSA.

4.8 Implementazioni di Cold Storage

Le soluzioni di cold storage hanno sviluppato varie implementazioni con diversi profili di sicurezza e convenienza:

- **Hardware wallet:** Dispositivi specializzati come Ledger, Trezor e BitBox che implementano protocolli HD con isolamento delle chiavi private.

- **Paper wallet:** Trascrizione fisica del seed, spesso integrata con meccanismi di cifratura o divisione in parti per sicurezza aggiuntiva.
- **Brain wallet:** Memorizzazione del seed o di una passphrase da cui derivare il seed, esposta a rischi di dimenticanza e attacchi a dizionario.
- **Multi-signature:** Configurazioni che richiedono multiple chiavi per autorizzare transazioni, aumentando la resilienza contro compromissioni di singoli punti.

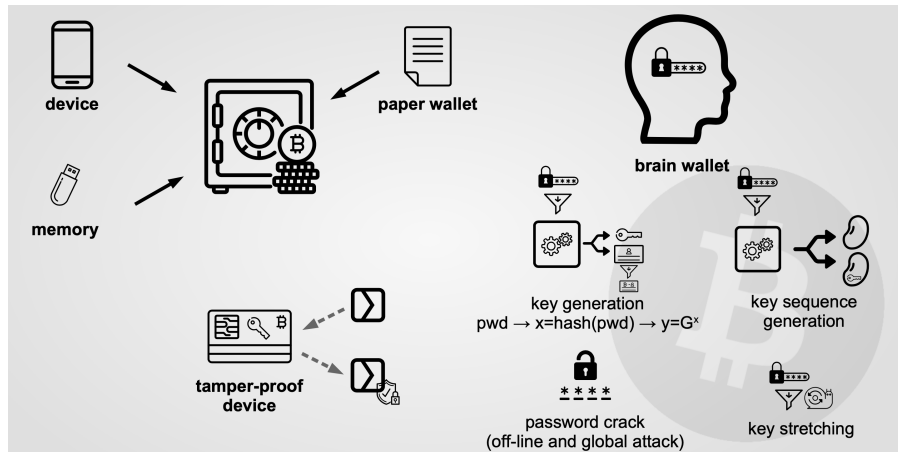


Figure 4.6: Comparazione delle diverse implementazioni di cold storage, con illustrazione di hardware wallet, paper wallet, e sistemi multi-signature.

Ciascuna di queste implementazioni offre un diverso bilanciamento tra sicurezza, complessità e recuperabilità, richiedendo una valutazione attenta in base al profilo di rischio dell'utente.

4.8.1 Considerazioni sulla Sicurezza delle Passphrase

L'uso di passphrase per derivare o proteggere le chiavi introduce vulnerabilità specifiche:

- **Attacchi di forza bruta:** Tentativi sistematici di indovinare la passphrase attraverso esplorazione esaustiva dello spazio delle possibilità.
- **Attacchi a dizionario:** Tentativi basati su liste di parole comuni, frasi, o variazioni prevedibili.
- **Key stretching:** Tecniche come PBKDF2, scrypt o Argon2 che aumentano artificialmente il costo computazionale della derivazione delle chiavi dalla passphrase.

La sicurezza effettiva di un sistema basato su passphrase dipende criticamente dall'entropia della passphrase stessa e dalla robustezza delle funzioni di derivazione utilizzate.

4.9 Secret Sharing e Threshold Cryptography

La gestione sicura delle chiavi private può beneficiare di tecniche di condivisione del segreto, che permettono di distribuire il rischio tra più componenti.

4.9.1 Principi del Secret Sharing

Il secret sharing è una tecnica crittografica che permette di dividere un segreto (come una chiave privata) in più parti, dette "shares", in modo che:

- Il segreto originale possa essere ricostruito solo combinando un numero sufficiente di shares.
- Nessuna informazione sul segreto sia rivelata analizzando un numero insufficiente di shares.

Implementazioni comuni includono:

- **XOR sharing:** $S = S_1 \oplus S_2$, dove il segreto S è diviso in due shares che, combinate con l'operazione XOR, riproducono il segreto originale.
- **Additive sharing:** $S = S_1 + S_2 + \dots + S_n \pmod{p}$, dove il segreto è diviso in n shares che sommate modulo p riproducono il segreto originale.

4.9.2 Threshold Secret Sharing

Lo schema di Shamir's Threshold Secret Sharing (t -out-of- n) rappresenta un'evoluzione che permette di specificare il numero minimo di shares necessario per la ricostruzione:

- Il segreto S viene incorporato come termine costante di un polinomio di grado $t-1$: $P_S(x) = S + a_1x + \dots + a_{t-1}x^{t-1} \pmod{p}$, dove i termini a_i sono numeri random e p è un numero primo sufficientemente grande da contenere il segreto S .
- Vengono generate n shares, ciascuna come coppia $(i, s_i = P_S(i))$ con $i \in [1, n]$
- Il segreto può essere ricostruito attraverso interpolazione polinomiale usando almeno t shares: $S = P_S(0) = \sum_{j=1}^t \lambda_j s_j \pmod{p}$, dove i λ_j sono i coefficienti di Lagrange.

La sicurezza di questo schema deriva dal fatto che, matematicamente, se abbiamo meno di t shares, esistono infiniti polinomi possibili di grado $t-1$ che passano per quei punti, ciascuno con un termine costante diverso. Questo significa che qualsiasi valore di segreto è ugualmente probabile, rendendo impossibile indovinarlo senza avere tutte le t shares necessarie.

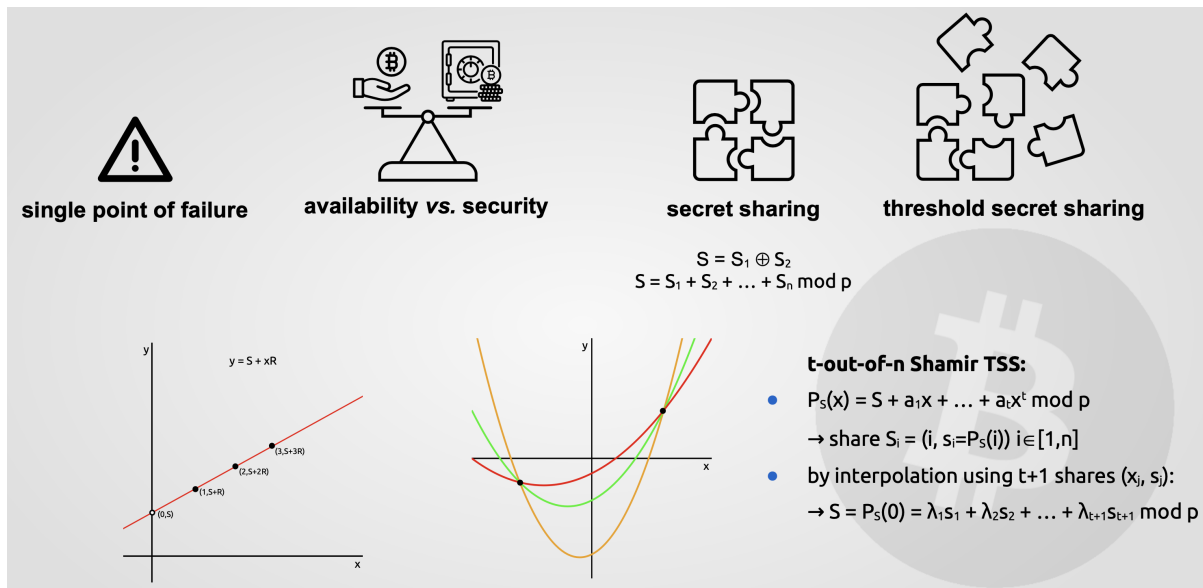


Figure 4.7: Rappresentazione visiva dello Shamir's Threshold Secret Sharing, mostrando il polinomio sottostante e il processo di interpolazione per la ricostruzione del segreto.

4.9.3 Threshold Signatures

L'implementazione di firme a soglia (threshold signatures) estende il concetto di secret sharing al processo di firma digitale:

- La chiave privata x viene condivisa tra n partecipanti usando threshold secret sharing.
- Una firma valida può essere generata solo con la collaborazione di almeno t partecipanti.
- La chiave privata non viene mai ricostruita completamente durante il processo di firma, riducendo il rischio di esposizione.

L'integrazione di threshold signatures con Bitcoin rappresenta una sfida tecnica significativa, specialmente per ECDSA che non è naturalmente compatibile con operazioni distribuite. Algoritmi di firma alternativi come Schnorr, introdotti con il soft fork Taproot, offrono proprietà matematiche più favorevoli per implementazioni di threshold signatures efficienti.

4.10 Bitcoin Exchanges

Gli exchange di criptovalute operano come intermediari che facilitano il trading e la conversione tra bitcoin e altre valute:

- **Funzionalità:** Offrono mercati di scambio, strumenti di trading e, spesso, servizi di custodia integrati.
- **Liquidità:** Aggregano domanda e offerta, facilitando transazioni di volumi significativi.

- **Gateway fiat:** Rappresentano spesso il punto di ingresso principale per nuovi utenti attraverso la conversione di valute tradizionali.

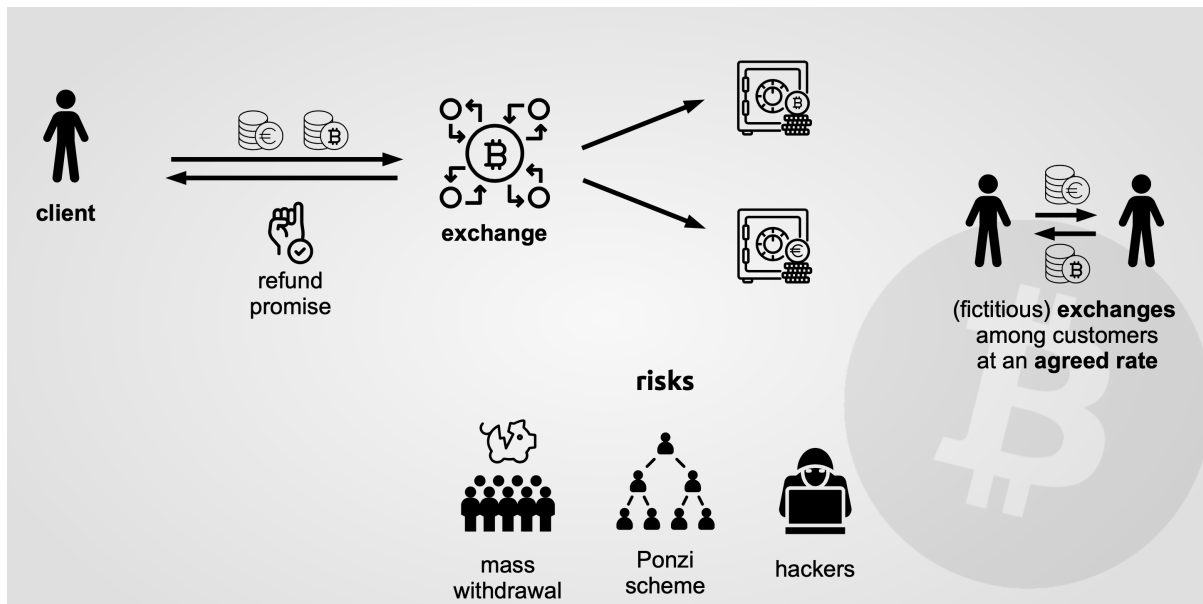


Figure 4.8: Modello operativo di un exchange Bitcoin, con illustrazione dei flussi di fondi tra clienti e dei meccanismi di riserva frazionaria.

4.10.1 Proof of Reserves

La trasparenza delle riserve rappresenta un meccanismo per mitigare il rischio di custodia negli exchange:

- **Proof of Reserves:** Dimostrazione crittografica che l'exchange controlla bitcoin sufficienti a coprire i saldi dei clienti.
- **Proof of Liabilities:** Verifica che i saldi dichiarati corrispondano effettivamente agli impegni verso i clienti.
- **Verifica di inclusione:** Meccanismo che permette ai clienti di confermare che i loro saldi sono stati inclusi nel calcolo delle passività totali.

Queste tecniche forniscono un lower bound della riserva frazionaria $F = \frac{R}{D} \geq \frac{r}{d} = f$, dove R rappresenta le riserve, D le passività, e r e d sono i rispettivi lower bound e upper bound verificabili.

Nonostante questi meccanismi di trasparenza, il principio "not your keys, not your coins" rimane fondamentale nell'ethos Bitcoin, sottolineando l'importanza del controllo diretto delle chiavi private per la vera proprietà dei fondi.

Chapter 5

Lezione 5

5.1 Il Processo di Mining

Il mining rappresenta il processo fondamentale attraverso cui nuovi blocchi vengono validati e aggiunti alla blockchain Bitcoin, combinando la funzione di emissione di nuova valuta con quella di sicurezza e consenso distribuito.

5.1.1 Attività del Miner

Un miner Bitcoin svolge diverse funzioni essenziali per il mantenimento della rete:

- **Verifica delle transazioni:** I miner convalidano le transazioni in ingresso secondo le regole di consenso della rete.
- **Mantenimento della blockchain:** Conservano una copia completa della blockchain e verificano la validità dei nuovi blocchi proposti.
- **Creazione di nuovi blocchi:** Selezionano un insieme di transazioni valide dal loro mempool e le organizzano in un candidato blocco.
- **Proof-of-Work:** Ricercano un valore di nonce che, combinato con i dati del blocco, produca un hash che soddisfi la difficoltà corrente.
- **Propagazione dei blocchi:** Una volta trovato un blocco valido, lo trasmettono immediatamente alla rete per massimizzare la probabilità di inclusione nella catena principale.

5.1.2 Difficoltà di Mining

Il parametro di difficoltà regola il livello di complessità del puzzle crittografico che i miner devono risolvere:

- **Target hash:** Un valore soglia che l'hash del blocco deve essere inferiore per essere considerato valido.
- **Rappresentazione:** Il target è espresso in notazione compatta nel campo "bits" dell'header del blocco.

- **Relazione con difficoltà:** La difficoltà è inversamente proporzionale al target: $\text{difficoltà} = \frac{\text{target_base}}{\text{target}}$, dove `target_base` è `0xFFFF` $\ll$ 208 (approssimativamente 2^{224}).

Ad esempio, per il blocco 839351 (aprile 2024):

- Difficoltà: 86,388,558,925,171.02
- Target approssimativo: 2^{178}
- Hash header: 0000000000000000000024...72cffe (inizia con 78 bit nulli)

Questa difficoltà implica che, in media, è necessario calcolare 2^{78} hash distinti prima di trovare un valore che soddisfi il requisito del target.

5.1.3 Aggiustamento Dinamico della Difficoltà

Per mantenere un intervallo medio di 10 minuti tra blocchi successivi, la difficoltà viene ricalibrata periodicamente:

- **Frequenza:** Ogni 2016 blocchi (approssimativamente ogni 2 settimane)
- **Formula:** $\text{difficoltà_successiva} = \text{difficoltà_precedente} \times \frac{2016 \times 10 \text{ minuti}}{\text{tempo effettivo per minare 2016 blocchi}}$

Questo meccanismo di feedback automatico garantisce che la frequenza di generazione dei blocchi rimanga relativamente costante, indipendentemente dalle fluttuazioni della potenza di calcolo totale della rete.

5.2 Tecniche di Mining

Il mining di Bitcoin ha subito un'evoluzione tecnologica drammatica dalla sua introduzione, transitando da un'attività accessibile su hardware consumer a un'industria specializzata con hardware dedicato.

5.2.1 Mining con CPU

Il mining su processori generici rappresenta l'approccio originale, ora completamente obsoleto:

- **Hashrate:** Circa 20 MH/s su processori moderni.
- **Efficienza:** Estremamente bassa per watt consumato.
- **Probabilità di successo:** Con la difficoltà attuale, un singolo PC moderno impiegherebbe in media circa 960 milioni di anni per minare un blocco.

L'algoritmo di base per il mining su CPU segue questa struttura:

```
1 target = (0xFFFF << 208) / difficulty;
2 coinbase_nonce = 0;
3 while (1) {
4     header = makeBlockHeader(transactions, coinbase_nonce);
5     for (header_nonce = 0; header_nonce < (1 << 32); header_nonce++) {
6         if (SHA256(SHA256(makeBlock(header, header_nonce))) < target)
7             break; // blocco trovato!!!
8     }
9     coinbase_nonce++;
10 }
```

5.2.2 Mining con GPU

Le unità di elaborazione grafica hanno rappresentato la prima evoluzione significativa nell'hardware di mining:

- **Parallelismo:** Le GPU offrono migliaia di core che possono eseguire operazioni in parallelo.
- **Programmabilità:** Ambienti di sviluppo come CUDA e OpenCL permettono l'ottimizzazione degli algoritmi di mining.
- **Hashrate:** Circa 200 MH/s per schede di fascia alta.
- **Limitazioni:**
 - Utilizzo parziale delle risorse della GPU (componenti non ottimizzati per hash SHA-256).
 - Efficienza termica subottimale.
 - Elevato consumo energetico.

5.2.3 Mining con FPGA

I Field Programmable Gate Array hanno rappresentato una fase intermedia nell'evoluzione dell'hardware di mining:

- **Circuiti logici generici:** Hardware riprogrammabile che può essere configurato specificamente per l'algoritmo SHA-256.
- **Linguaggi HDL:** Programmazione attraverso linguaggi come Verilog o VHDL, che permettono descrizioni hardware di basso livello.
- **Hashrate:** Circa 1 GH/s per dispositivo.
- **Efficienza:** Significativamente migliore rispetto a CPU e GPU, ma ancora inferiore agli ASIC.

La fase FPGA è stata relativamente breve nella storia del mining Bitcoin, rapidamente superata dagli ASIC più specializzati e efficienti.

5.2.4 Mining con ASIC

Gli Application Specific Integrated Circuit rappresentano lo stato dell'arte attuale per il mining Bitcoin:

- **Circuiti dedicati:** Progettati specificamente per eseguire l'algoritmo SHA-256 con massima efficienza.
- **Hashrate:** Da 40 TH/s a oltre 100 TH/s per dispositivo moderno.
- **Vantaggi:**
 - Prestazioni estreme rispetto alle altre soluzioni.
 - Efficienza energetica ottimizzata per watt consumato.
- **Svantaggi:**
 - Rapida obsolescenza con l'evoluzione tecnologica.
 - Utilizzo specifico limitato al mining di criptovalute basate su SHA-256.

Esempi di ASIC moderni includono:

- Antminer S19j Pro: 110 TH/s, circa 3.000€
- Antminer T17: 42 TH/s, circa 800€

5.3 Mining Pool

I mining pool rappresentano una soluzione di collaborazione che permette ai miner individuali di combinare la loro potenza di calcolo per ridurre la varianza nei guadagni.

5.3.1 Principio di Funzionamento

Un mining pool coordina l'attività di numerosi miner individuali:

- **Lavoro distribuito:** Il pool assegna a ciascun partecipante un intervallo specifico di nonce da esplorare.
- **Shares:** I miner inviano "share" al pool, che sono soluzioni parziali (hash che soddisfano un target meno stringente del target di rete).
- **Validazione centralizzata:** Il pool coordina e verifica le shares ricevute.
- **Ricompensa proporzionale:** Quando un miner del pool trova un blocco valido, la ricompensa viene distribuita tra i partecipanti in proporzione alle shares contribute.

5.3.2 Modelli di Distribuzione della Ricompensa

I pool implementano diversi schemi per la distribuzione delle ricompense:

- **Pay-per-Share (PPS):** Pagamento fisso per ogni share valida, indipendentemente dal successo del pool nel trovare blocchi.
- **Proportional:** Distribuzione proporzionale al numero di shares contribute durante il round (periodo tra due blocchi trovati).

- **PPLNS (Pay Per Last N Shares)**: Simile al proportional, ma considera solo le ultime N shares prima del blocco trovato.
- **Score-based**: Assegnazione di punteggi alle shares che diminuiscono con il tempo, favorendo la fedeltà al pool.

Ogni modello presenta diversi profili di rischio e ricompensa, con trade-off tra prevedibilità dei pagamenti e potenziale guadagno a lungo termine.

Chapter 6

Capitolo 6

6.1 Dal Contratto Tradizionale allo Smart Contract

6.1.1 Paradigmi Contrattuali a Confronto

I contratti tradizionali e gli smart contracts rappresentano due paradigmi fondamentalmente diversi per la formalizzazione e l'esecuzione di accordi tra parti. I contratti tradizionali, talvolta ironicamente definiti "dumb contracts" nella letteratura specialistica, si caratterizzano per la loro natura interpretativa e la dipendenza da intermediari. Questi contratti richiedono la stesura da parte di professionisti del diritto e sono soggetti a interpretazione umana, con possibili ambiguità che possono portare a controversie. In caso di disaccordo, è necessario l'intervento di un giudice o altro arbitro per risolvere la disputa. Le clausole contrattuali vengono imposte attraverso il sistema legale, che fornisce il meccanismo di enforcement esterno.

In netto contrasto, gli smart contracts si basano su un approccio deterministico e auto-esecutivo. Sviluppati da programmatori anziché avvocati, offrono un'interpretazione non ambigua delle condizioni contrattuali, poiché il codice stesso definisce completamente le regole e le conseguenze. La frase "the code says everything" sintetizza efficacemente questo concetto: il codice è simultaneamente la definizione del contratto, la sua interpretazione e il suo meccanismo di esecuzione. Gli smart contracts non necessitano di terze parti per l'applicazione delle clausole, in quanto l'esecuzione è automatica e garantita dalle proprietà crittografiche della blockchain sottostante. Inoltre, essendo implementati su piattaforme blockchain pubbliche, sono caratterizzati da trasparenza e accessibilità del codice, permettendo a tutte le parti interessate di verificare le esatte condizioni di esecuzione.

Questa fondamentale differenza tra i due paradigmi non è meramente tecnica, ma rappresenta una visione profondamente diversa del concetto stesso di accordo. Il passaggio dall'enforcement legale all'auto-esecuzione algoritmica costituisce una trasformazione radicale dei meccanismi di fiducia: dalla fiducia nelle istituzioni alla fiducia nell'algoritmo e nella crittografia.

6.1.2 Limiti di Bitcoin Script ed Evoluzione Necessaria

Bitcoin, pur essendo la prima implementazione di blockchain con capacità di scripting, presenta limitazioni significative che ne restringono l'applicabilità per smart contracts complessi. Il linguaggio di scripting di Bitcoin è stato intenzionalmente progettato con vincoli che ne limitano la flessibilità per ragioni di sicurezza e determinismo.

Le principali limitazioni di Bitcoin Script sono:

- **No loops:** L'assenza di strutture iterative (cicli) per prevenire loop infiniti durante la validazione.
- **Whitelist:** Un insieme limitato di operazioni consentite, riducendo i rischi di vulnerabilità ma anche le potenzialità espressive.
- **Limited logic:** Capacità logiche limitate che impediscono implementazioni complesse.
- **Blockchain blindness:** Incapacità di accedere a informazioni esterne alla specifica transazione in esecuzione.
- **Stateless:** Impossibilità di memorizzare e accedere a stati persistenti tra diverse transazioni, limitando la creazione di smart contract complessi che richiedono il mantenimento di uno stato.

Queste restrizioni, sebbene motivate da considerazioni di sicurezza, rendono il sistema non Turing-completo, impedendo l'implementazione di logiche contrattuali complesse. La natura stateless degli script Bitcoin, privi di memoria persistente tra le transazioni, rende impossibile mantenere uno stato interno nel tempo, caratteristica essenziale per contratti che evolvono nel tempo. Inoltre, la blockchain blindness impedisce di fatto la creazione di contratti interdipendenti o basati su condizioni complesse che richiedono l'accesso a informazioni globali.

L'effetto combinato di queste limitazioni ha reso Bitcoin inadeguato per implementare applicazioni decentralizzate sofisticate, creando la necessità di piattaforme blockchain alternative specificamente progettate per supportare smart contracts avanzati.

6.1.3 NameCoin: Un Caso di Studio Emblematico

NameCoin rappresenta un caso illuminante dei tentativi iniziali di estendere le funzionalità di Bitcoin per implementare servizi decentralizzati più complessi. Concepito come sistema di registrazione di nomi sulla blockchain, NameCoin aveva l'obiettivo di associare nomi di dominio (domain name) a indirizzi IP, creando un sistema DNS resistente alla censura.

```

OP_DUP
OP_HASH256
<ownAddr>
OP_EQVERIFY
OP_CHECKSIG
OP_VERIFY
<NameCoin>
<domainName>
<ipAddr>
<1>

```

Il sistema supportava tre operazioni fondamentali:

- `NameCoin.register(ownerAddr, domainName, initialVal)`: per registrare un nuovo dominio
- `NameCoin.update(domainName, newVal, newOwner, ownerSig)`: per aggiornare informazioni o trasferire proprietà
- `NameCoin.lookup(domainName)`: per ottenere le informazioni associate a un dominio

Il tentativo di implementare NameCoin utilizzando Bitcoin Script ha rivelato un limite critico del paradigma: l'impossibilità di imporre la politica fondamentale secondo cui "un dominio già registrato non può essere registrato nuovamente". Questa regola, essenziale per un sistema di registrazione di nomi, non poteva essere implementata con gli script Bitcoin a causa della loro natura stateless e dell'impossibilità di accedere allo stato globale della blockchain.

Il team dietro NameCoin fu costretto a sviluppare una soluzione radicale: la creazione di una blockchain completamente nuova (namecoin.org), con una propria criptovaluta (NMC) e un protocollo modificato. Questo approccio ha permesso l'implementazione di domini con estensione .bit, validazione di certificati TLS e resistenza alla censura. Tuttavia, la necessità di un nuovo bootstrap e di costruire una comunità dedicata ha rappresentato una sfida significativa per l'adozione della tecnologia.

Questo caso illustra chiaramente il costo della frammentazione dell'ecosistema blockchain: ogni nuova blockchain richiede l'allocazione di risorse computazionali separate per il mining, la creazione di una nuova economia tokenizzata, e lo sviluppo di un'infrastruttura dedicata. L'esempio di NameCoin ha evidenziato la necessità di una piattaforma blockchain generale capace di implementare smart contracts avanzati senza richiedere la creazione di nuove blockchain specializzate per ogni applicazione.

6.2 La Piattaforma Ethereum: Concetti Fondanti

6.2.1 Visione e Applicazioni

Ethereum rappresenta una risposta diretta alle limitazioni emerse nelle prime blockchain, concepita fin dall'origine come una piattaforma generale per l'esecuzione di smart contracts e applicazioni decentralizzate (DApps). La visione fondamentale di Ethereum,

articolata dal suo creatore Vitalik Buterin, era di realizzare un "computer mondiale" decentralizzato, capace di eseguire programmi arbitrari in un ambiente trustless.

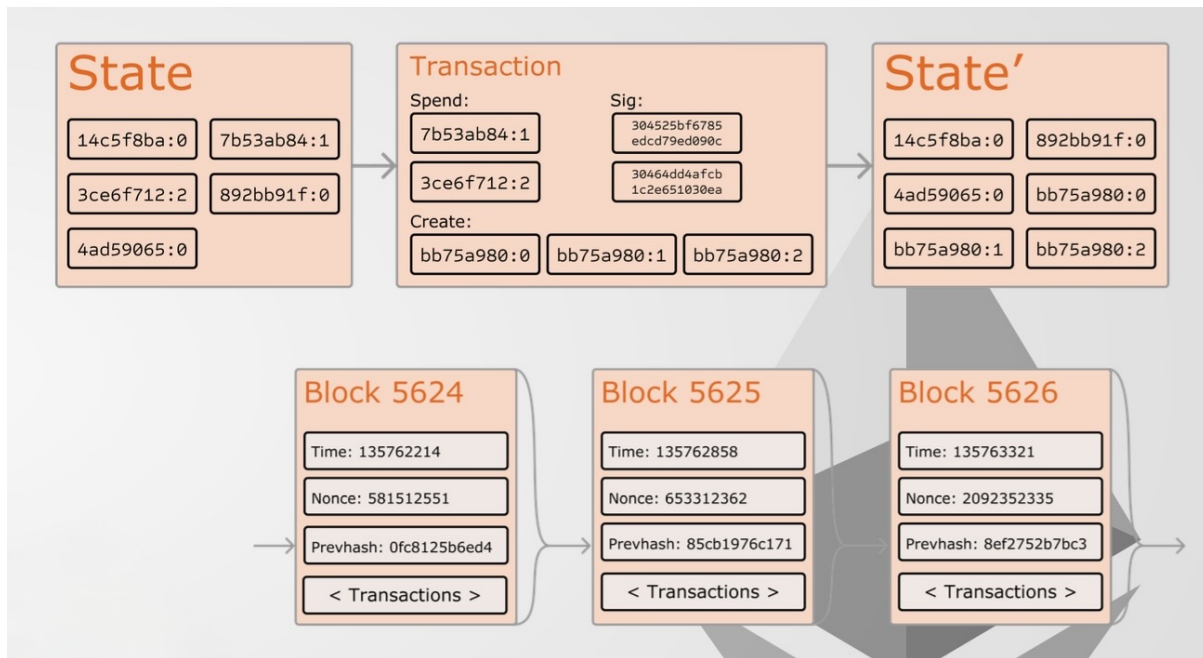
Questa visione si è concretizzata in un ecosistema vibrante di applicazioni, ciascuna delle quali sfrutta le capacità uniche della piattaforma:

- **Token personalizzati (ERC-20):** Lo standard ERC-20 ha democratizzato l'emissione di asset digitali definendo un'interfaccia comune per funzionalità come trasferimenti, approvazioni e bilanci. Questo ha abilitato la creazione di migliaia di token con diverse utility, dai token di governance ai token di utilità fino ai stable coin.
- **Prodotti finanziari decentralizzati:** L'ecosistema DeFi (Finanza Decentralizzata) ha implementato componenti fondamentali del sistema finanziario - prestiti, scambi, derivati, assicurazioni - senza intermediari tradizionali, creando un sistema finanziario parallelo accessibile globalmente.
- **NFT (ERC-721):** I Non-Fungible Token hanno rivoluzionato la rappresentazione della proprietà digitale, permettendo l'attribuzione univoca e verificabile di asset digitali e creando nuovi mercati per arte digitale, collezionabili virtuali e altre forme di proprietà unica.
- **Organizzazioni Autonome Decentralizzate (DAO):** Le DAO implementano governance e processi decisionali attraverso smart contracts, consentendo forme organizzative completamente digitali con regole trasparenti e auto-esecutive.
- **Gaming e metaverso:** Applicazioni come CryptoKitties e Sorare hanno dimostrato il potenziale dei giochi basati su blockchain, integrando proprietà digitale verificabile e economie virtuali decentralizzate.

L'ecosistema Ethereum è supportato da un'infrastruttura informativa robusta, con risorse come ethereum.org per la documentazione ufficiale, etherscan.io ed ethstats.net per l'analisi blockchain, e dapp.review e stateofthedapps.com per la scoperta di applicazioni. Questa infrastruttura facilita l'adozione e lo sviluppo, riducendo le barriere d'ingresso per sviluppatori e utenti.

6.2.2 Architettura Concettuale: Ethereum vs Bitcoin

Per comprendere appieno l'innovazione rappresentata da Ethereum, è illuminante confrontarla con Bitcoin, evidenziando le divergenze architetturali che riflettono obiettivi fondamentalmente diversi.



Bitcoin implementa una blockchain basata sulle transazioni, dove i saldi sono derivati indirettamente attraverso l'analisi delle transazioni non spese (UTXO). L'enfasi primaria è sulla funzione di valuta digitale, con capacità di smart contracting limitate. Il consenso è basato su Proof-of-Work (PoW) con funzione hash SHA-256, generando blocchi approssimativamente ogni 10 minuti. Con l'evoluzione dell'ecosistema, Bitcoin ha affrontato tendenze alla centralizzazione attraverso la predominanza di ASIC specializzati e mining pool, che concentrano la potenza di calcolo.

Ethereum, d'altra parte, concettualizza lo stato della rete come una collezione di stati di account, distinguendo tra:

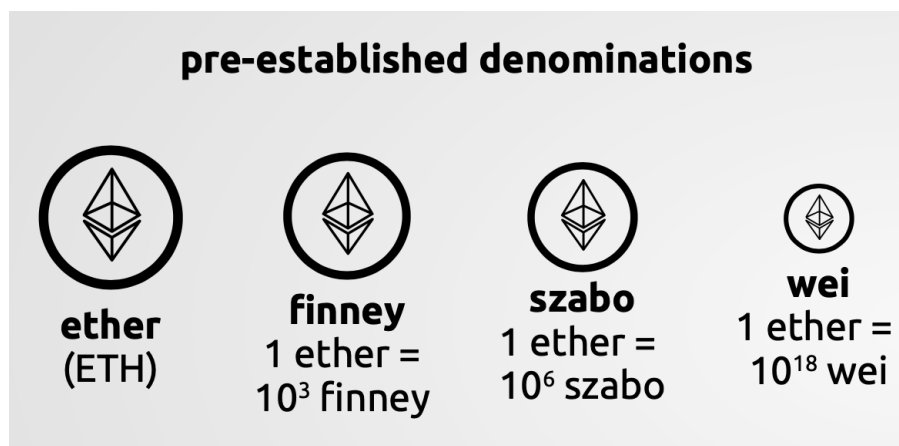
- **Account classici (EOA):** Controllati da utenti tramite chiavi private, contengono saldi in ETH.
- **Account contrattuali:** Entità autonome con codice immutabile e stato interno persistente, che possono contenere sia ETH che informazioni arbitrarie.

Le transizioni di stato sono indotte da transazioni raggruppate in blocchi, con particolare enfasi sulle transazioni che attivano la logica contrattuale. Originariamente basato su PoW con algoritmo ethash (Keccak-256), Ethereum ha completato nel settembre 2022 la transizione verso Proof-of-Stake (PoS) con "The Merge", affrontando questioni di scalabilità ed impatto ambientale.

Questa differenza architetturale da un modello basato su transazioni a un modello basato su account è fondamentale. Ethereum non è solo un registro di transazioni, ma un sistema di stato completo che evolve attraverso transizioni determinate dall'esecuzione di codice. Questa concezione ha permesso l'implementazione di contratti con stato persistente, condizione necessaria per applicazioni complesse.

6.2.3 La Valuta Nativa: Ether (ETH)

Ether (ETH), la valuta nativa di Ethereum, è progettata con un duplice ruolo: mezzo di scambio di valore e "carburante" computazionale per l'esecuzione di contratti. Questa dualità funzionale è una caratteristica distintiva rispetto a Bitcoin, concepito principalmente come riserva di valore e mezzo di pagamento.



Le denominazioni di ETH seguono una struttura decimale predefinita:

- 1 ether = 10^3 finney
- 1 ether = 10^6 szabo
- 1 ether = 10^{18} wei

Il wei rappresenta l'unità atomica indivisibile, analoga al satoshi in Bitcoin, ed è l'unità utilizzata a livello di protocollo per specificare valori nelle transazioni e calcolare costi computazionali.

6.2.4 Proof of Stake

Il Proof of Stake rappresenta un'evoluzione fondamentale nei meccanismi di consenso blockchain, sostituendo l'intenso lavoro computazionale del Proof of Work con un sistema basato sull'impegno economico dei partecipanti. In questo modello, i validatori bloccano una quantità della criptovaluta nativa come garanzia ("stake"), ottenendo il diritto di verificare le transazioni e creare nuovi blocchi in proporzione al capitale impegnato. I vantaggi di questo approccio sono sostanziali: riduzione drastica del consumo energetico, eliminazione della corsa agli hardware specializzati, e creazione di un sistema in cui la sicurezza della rete è direttamente legata al suo valore economico. La solidità del sistema è garantita da meccanismi di penalizzazione che colpiscono i validatori che violano le regole, confiscando parte del loro deposito. Il principale problema teorico, noto come "Nothing at Stake", viene contrastato attraverso penalizzazioni per chi valida blocchi concorrenti e sistemi che rendono definitivi i blocchi confermati dalla maggioranza qualificata dei validatori. La transizione di Ethereum verso questo modello, completata nel 2022, ha dimostrato che il PoS può sostenere efficacemente anche blockchain con elevato valore economico, aprendo la strada a reti più efficienti, accessibili e sostenibili.

6.2.5 Mining e Validazione: Dal Proof-of-Work al Proof-of-Stake

Prima della transizione a Proof-of-Stake con The Merge, il sistema di mining di Ethereum operava secondo un paradigma Proof-of-Work con caratteristiche distintive progettate per mitigare le tendenze centralizzanti osservate in Bitcoin.

L'algoritmo ethash utilizzava la funzione hash Keccak-256 (simile ma non identica a SHA-3) con specifiche contromisure contro l'uso di ASIC. In particolare, il design memory-hard di ethash rendeva il mining efficiente su GPU ma inefficiente su hardware specializzato, promuovendo una maggiore decentralizzazione. Questo approccio si è tradotto in una distribuzione più equa del potere di mining rispetto a Bitcoin.

Il tempo target tra blocchi era significativamente più basso rispetto a Bitcoin, attestandosi intorno ai 12 secondi. Questa maggiore frequenza di blocchi permetteva conferme più rapide ma comportava un tasso più elevato di blocchi orfani (uncle blocks), compensati attraverso ricompense parziali per mantenere l'equità del sistema.

La ricompensa originale per blocco era di 5 ETH, successivamente ridotta a 3 ETH e infine a 2 ETH prima della transizione a PoS. Questa ricompensa era integrata dalle commissioni di transazione (gas), creando un duplice incentivo economico per i miner.

La transizione al Proof-of-Stake con The Merge ha rappresentato una trasformazione fondamentale:

- Eliminazione del mining computazionale, sostituito dal meccanismo di staking che richiede il deposito di ETH come garanzia.
- Riduzione drastica del consumo energetico (>99.9%).
- Cambiamento nel modello di sicurezza, dalla sicurezza basata su costo energetico alla sicurezza basata su incentivi economici e slashing (penalizzazioni per comportamenti malevoli).

Questa transizione storica ha dimostrato la capacità di Ethereum di evolvere significativamente pur mantenendo la continuità operativa della rete, evidenziando il potenziale di governance e aggiornabilità delle blockchain pubbliche.

Chapter 7

Capitolo 7

7.1 Struttura degli Account e Gestione dello Stato

7.1.1 Tipologia e Proprietà degli Account

Il modello basato su account di Ethereum differisce sostanzialmente dall'approccio UTXO di Bitcoin, costituendo uno dei fondamenti architetturali distintivi della piattaforma. Esistono due tipologie di account nel sistema, con caratteristiche e funzionalità complementari.

associated data	owned	contract
nonce	✓	✓
address	hash(pk)	hash(creatorAddr, creatorNonce)
balance	✓	✓
codeHash	✗	✓
storageRoot	✗	✓

Gli **External Owned Account (EOA)** sono controllati da entità esterne attraverso coppie di chiavi crittografiche. Ogni EOA possiede:

- **Indirizzo:** Identificatore univoco derivato dall'hash della chiave pubblica (160 bit).
- **Firma ECDSA:** Meccanismo di autenticazione basato sulla curva ellittica secp256k1 (la stessa di Bitcoin).
- **Nonce:** Contatore incrementale che traccia il numero di transazioni inviate dall'account, prevenendo replay attack.
- **Saldo:** Quantità di ETH posseduta dall'account.

Solo gli EOA possono iniziare transazioni, richiedendo un'autorizzazione esplicita mediante firma crittografica per ogni operazione.

I **Contract Account** rappresentano smart contracts deployati sulla blockchain. Questi account:

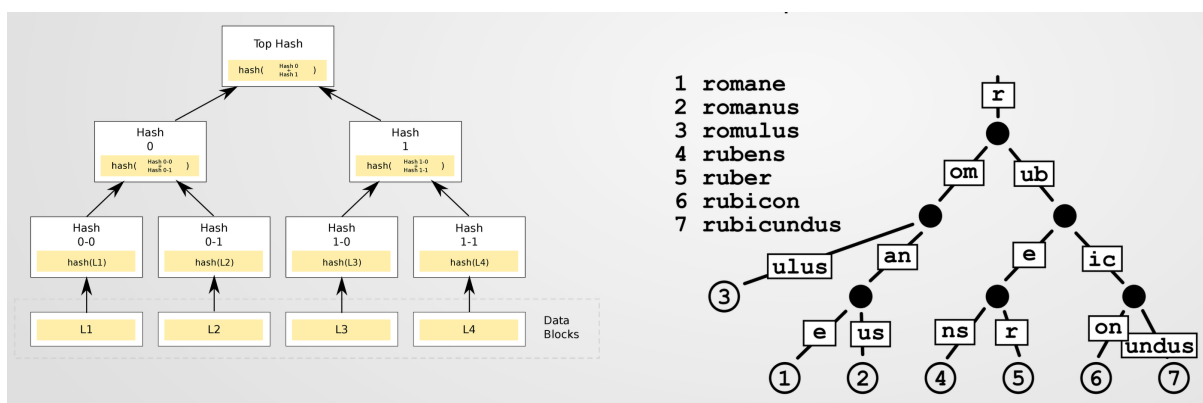
- Non possiedono chiavi private e quindi non possono iniziare transazioni autonomamente.
- Sono attivati esclusivamente in risposta a transazioni da EOA o messaggi da altri contratti.
- Hanno un indirizzo derivato deterministicamente dall'indirizzo del creatore e dal suo nonce al momento della creazione.
- Contengono, oltre a nonce e saldo, due elementi aggiuntivi:
 - **codeHash**: Impronta del codice immutabile del contratto.
 - **storageRoot**: Radice dell'albero Merkle Patricia che rappresenta lo stato interno persistente.

Il codice del contratto è immutabile: una volta deployato non può essere modificato, garantendo che le regole di interazione rimangano costanti. Lo storage interno, d'altra parte, è non volatile ma modificabile secondo le regole codificate nel contratto, permettendo l'evoluzione dello stato durante l'esecuzione.

Questa architettura duale permette l'interazione tra utenti e contratti, creando un ecosistema flessibile dove entità autonome (contratti) possono interagire con entità controllate dagli utenti (EOA), costituendo la base per la creazione di sistemi decentralizzati complessi.

7.1.2 Merkle Patricia Trie: Una Struttura Dati Ibrida

La gestione efficiente dello stato globale e dello storage dei contratti richiede una struttura dati sofisticata che bilanci verificabilità crittografica, efficienza di ricerca e ottimizzazione dello spazio. Ethereum implementa il Merkle Patricia Trie, una struttura ibrida che combina i vantaggi di due paradigmi distinti.



Il Merkle Patricia Trie integra le caratteristiche di due strutture dati complementari: l'albero di Merkle e il Patricia Trie. L'albero di Merkle è una struttura crittografica dove le foglie contengono hash di blocchi di dati, i nodi interni contengono hash calcolati dai loro figli, e la radice fornisce un'impronta univoca dell'intero dataset. Questa organizzazione

gerarchica di hash consente di verificare l'appartenenza di un elemento attraverso un percorso logaritmico di hash dalla foglia alla radice, chiamato "Merkle proof".

Il Patricia Trie (acronimo di "Practical Algorithm To Retrieve Information Coded In Alphanumeric") costituisce invece una variante ottimizzata dell'albero prefisso tradizionale. In questa struttura, ogni percorso dalla radice a una foglia rappresenta una chiave univoca, con compressione dei nodi singolarmente figli per ottimizzare lo spazio di archiviazione. Questa organizzazione permette ricerche efficienti nelle mappature chiave-valore, dove la complessità dipende dalla lunghezza della chiave anziché dal numero totale di elementi memorizzati.

La combinazione di queste strutture nel Merkle Patricia Trie crea un sistema con proprietà uniche fondamentali per Ethereum:

L'organizzazione delle chiavi definisce percorsi univoci nell'albero, dove ogni indirizzo Ethereum (per il World State) o indice di storage (per i contratti) corrisponde esattamente a un percorso dalla radice a una foglia. I nodi interni e le foglie sono identificati dai loro hash crittografici, e l'hash della radice rappresenta un'impronta unica dell'intero stato, memorizzata negli header dei blocchi.

Una caratteristica cruciale di questa struttura è la possibilità di effettuare modifiche efficienti: quando un valore viene aggiornato, solo i nodi lungo il percorso dalla foglia modificata alla radice devono essere ricalcolati. Questo permette aggiornamenti incrementali dello stato con costo logaritmico rispetto al numero di elementi, evitando di ricalcolare l'intero stato dopo ogni transazione. Poiché la funzione hash è unidirezionale, qualsiasi modifica produce necessariamente un nuovo hash di radice, garantendo l'integrità dell'intero sistema.

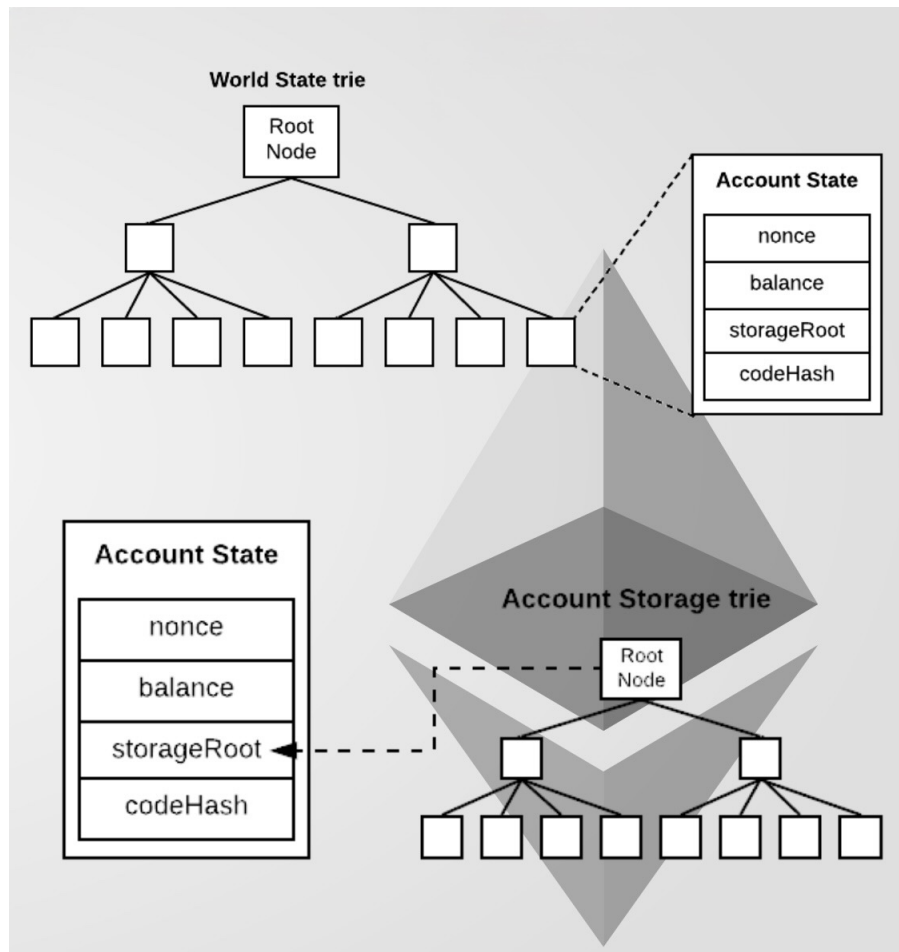
Per verificare l'appartenenza di un elemento (prova di appartenenza), è sufficiente fornire il valore stesso e i nodi fratelli lungo il percorso dalla foglia alla radice. Un verificatore, conoscendo solo l'hash della radice, può calcolare l'hash del valore, combinarlo con gli hash dei nodi fratelli secondo il percorso definito dalla chiave, e verificare che l'hash risultante corrisponda all'hash della radice. Questa verifica richiede operazioni proporzionali solo alla profondità dell'albero, non alla dimensione totale dello stato.

Questa architettura abilita i "light clients" in Ethereum: nodi che non mantengono l'intero stato ma possono comunque verificare transazioni richiedendo e validando solo le porzioni di stato rilevanti, senza compromettere la sicurezza crittografica. Il Merkle Patricia Trie costituisce quindi il fondamento per diverse strutture chiave: il **World State Trie** che mappa indirizzi a stati di account, lo **Storage Trie** per lo stato interno dei contratti, il **Transactions Trie** per le transazioni in un blocco, e il **Receipts Trie** per le ricevute di esecuzione.

Questa infrastruttura consente a Ethereum di funzionare come una "macchina a stati" globale e verificabile, dove ogni blocco rappresenta una transizione deterministica da uno stato all'altro, con garanzie crittografiche di integrità e immutabilità.

7.1.3 Organizzazione Gerarchica dello Stato

Lo stato complessivo di Ethereum è organizzato in una gerarchia di strutture Merkle Patricia Trie, creando un sistema di riferimenti che permette la verifica e l'accesso efficiente a qualsiasi aspetto dello stato.



Il **World State** rappresenta lo stato aggregato di tutti gli account sulla blockchain. Pur non essendo tecnicamente parte dei blocchi (che contengono solo transazioni e non stato), ogni nodo completo mantiene una copia aggiornata di questo stato in un database dedicato. Implementato come un Merkle Patricia Trie, mappa indirizzi (160 bit) ai rispettivi stati di account. L'**Account State** contiene le informazioni specifiche di ciascun account:

- Per EOA: nonce e balance.
- Per Contract Account: nonce, balance, codeHash e storageRoot.

Lo **Storage degli account contrattuali** è anch'esso implementato come Merkle Patricia Trie individuale per ciascun contratto. Concettualmente equivalente a un array di storage $S[\]$ con indici di 32 byte, mappa hash di 256 bit a valori di 256 bit. I valori sono codificati utilizzando RLP (Recursive Length Prefix), un formato di serializzazione efficiente per strutture dati annidate. Il valore predefinito per posizioni non inizializzate è 0. Questa architettura stratificata consente una gestione efficiente dello stato, con diverse proprietà fondamentali:

- **Verificabilità crittografica:** La radice del world state (stateRoot) inclusa nell'header del blocco permette di verificare crittograficamente qualsiasi aspetto dello stato senza necessità di fidarsi di terze parti.
- **Accesso selettivo:** È possibile accedere e verificare specifici elementi dello stato senza scaricare l'intero stato.

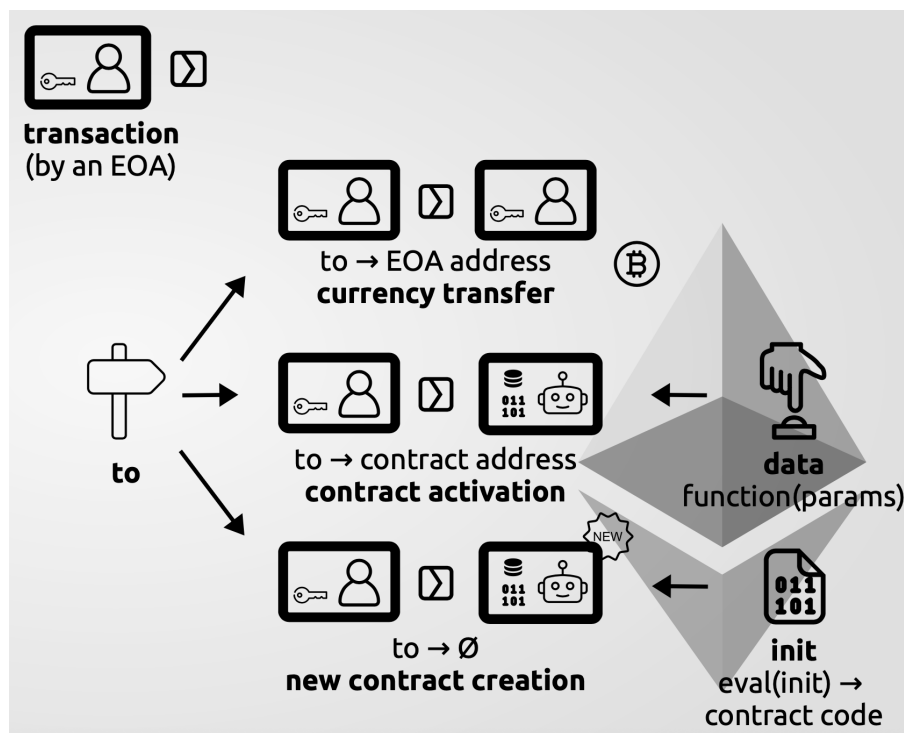
- **Aggiornamenti efficienti:** Modificare lo stato richiede solo il ricalcolo di un numero logaritmico di nodi nel trie, non dell'intera struttura.

Questa organizzazione costituisce la base tecnica che permette a Ethereum di funzionare come una "macchina a stati" globale e decentralizzata, dove ogni blocco rappresenta una transizione deterministica da uno stato all'altro.

7.2 Transazioni, Messaggi e Costruzione dei Blocchi

7.2.1 Anatomia delle Transazioni

Le transazioni in Ethereum costituiscono il meccanismo primario attraverso cui gli utenti interagiscono con la blockchain, modificando lo stato globale. Una transazione è sempre iniziata da un External Owned Account (EOA) e può svolgere diverse funzioni a seconda dei suoi campi.



La struttura di una transazione Ethereum include:

- **Firma digitale (v, r, s):** Prodotta con ECDSA (variante recuperabile) che autentica il mittente e garantisce l'integrità della transazione.
- **Valore:** Quantità di ETH trasferito, espresso in wei come intero.
- **Destinazione ("to"):** Può assumere tre forme:
 - Indirizzo EOA: Per trasferimenti diretti di valuta.
 - Indirizzo di contratto: Per attivare funzioni in un contratto esistente.
 - Vuoto ($\emptyset$): Per creare un nuovo contratto.

- **Data/Input:** Per transazioni dirette a contratti, contiene i parametri della funzione da invocare, codificati secondo l'Application Binary Interface (ABI).
- **Init:** Per la creazione di contratti, contiene il codice di inizializzazione che, quando eseguito, produce il codice definitivo del contratto.
- **Nonce:** Contatore incrementale specifico per l'account mittente, che previene replay attack e stabilisce l'ordine delle transazioni.
- **Gas:** Serie di campi che definiscono limiti e compensi per l'esecuzione computazionale (discussi in dettaglio nella sezione dedicata).

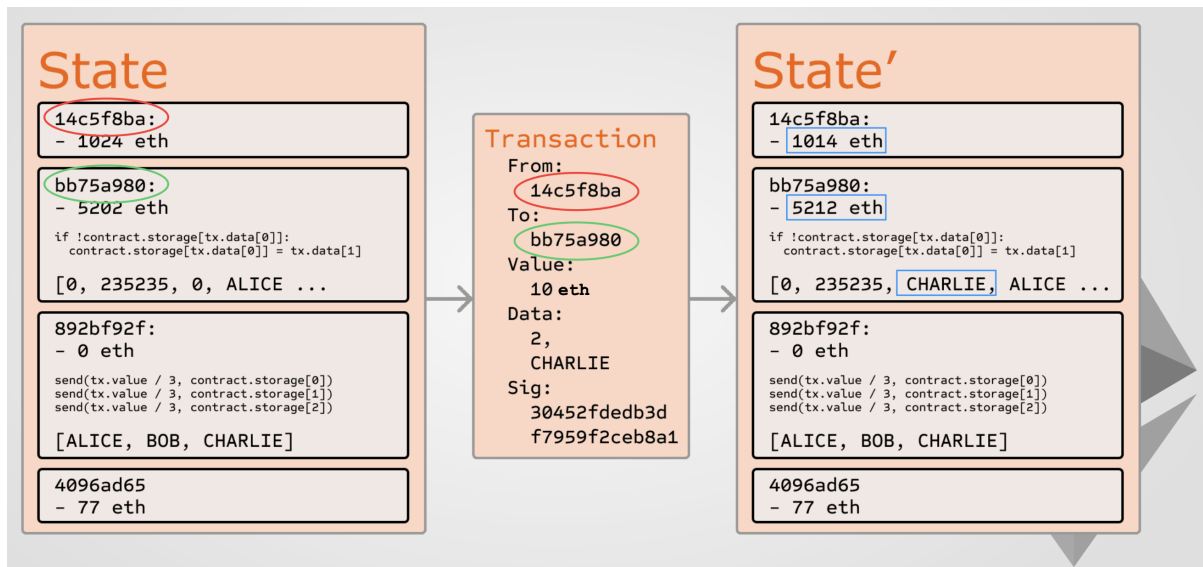


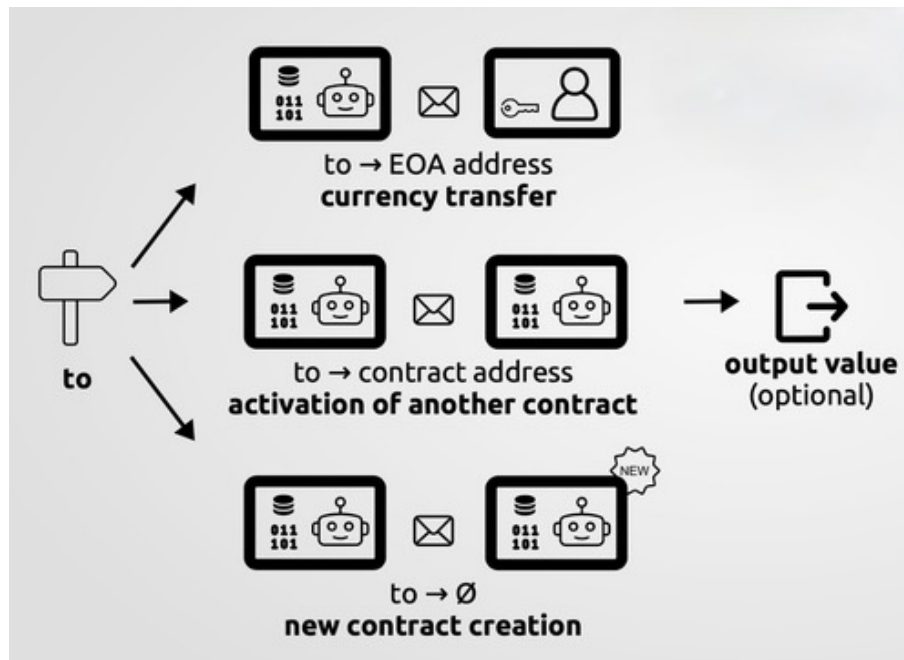
Figure 7.1: Un esempio semplificato di transazione.

Un esempio concreto di transazione evidenzia come questa struttura si traduca in pratica. In una transazione di trasferimento, lo stato pre-transazione mostra gli account mittente e destinatario con i rispettivi saldi. L'esecuzione della transazione modifica questi saldi, decrementando quello del mittente (dell'importo trasferito più le commissioni) e incrementando quello del destinatario.

Le transazioni reali, visualizzabili attraverso esploratori come Etherscan, rivelano la complessità completa di questi oggetti, includendo campi tecnici come gas price, gas limit, e nonce, oltre a firme crittografiche e rappresentazioni esadecimali dei dati.

7.2.2 Messaggi e Composabilità

Mentre le transazioni rappresentano l'interfaccia esterna attraverso cui gli utenti interagiscono con Ethereum, i messaggi costituiscono il meccanismo di comunicazione interna tra contratti, creando la base per la composabilità - una delle proprietà più potenti dell'ecosistema.



I messaggi sono comunicazioni generate dall'esecuzione di contratti, e presentano importanti differenze rispetto alle transazioni:

- **Autenticazione implicita:** Non necessitano di firme crittografiche, poiché l'autenticità è intrinsecamente garantita dall'ambiente di esecuzione controllato.
- **Versatilità operativa:** Possono trasferire valore a EOA, attivare altri contratti, o persino creare nuovi contratti.
- **Output:** Possono generare valori di ritorno utilizzabili dal contratto chiamante, creando flussi di dati bidirezionali.

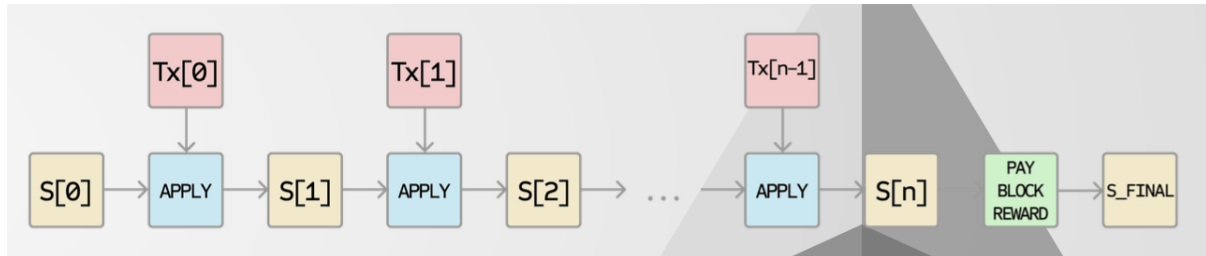
La composabilità - la capacità dei contratti di interagire tra loro in modo modulare - è una delle caratteristiche più rivoluzionarie di Ethereum. Una singola transazione iniziata da un EOA può generare una cascata di messaggi tra contratti, creando interazioni complesse che attraversano l'intero ecosistema. Questo permette la realizzazione del principio di "money legos", dove componenti finanziari semplici possono essere combinati per creare sistemi complessi e innovativi. Ad esempio, una singola transazione potrebbe attivare una sequenza di:

- Scambio di token su un exchange decentralizzato
- Deposito della liquidità ottenuta in un protocollo di lending
- Utilizzo del token di prestito come collaterale per un'altra operazione
- Distribuzione dei profitti secondo regole predefinite

Questa architettura composabile ha permesso la creazione di ecosistemi come DeFi, dove protocolli indipendenti possono interagire senza necessità di autorizzazioni o integrazioni esplicite, accelerando drammaticamente l'innovazione e la complessità del sistema.

7.2.3 Processo di Costruzione dei Blocchi

La creazione e validazione dei blocchi in Ethereum rappresenta il processo fondamentale attraverso cui le transazioni vengono incorporate permanentemente nella blockchain e lo stato globale viene aggiornato. Questo processo ha subito modifiche significative con la transizione da Proof-of-Work a Proof-of-Stake, ma i principi fondamentali restano simili.



Il processo di costruzione di un blocco da parte di un validatore (precedentemente miner) comprende:

1. **Raccolta di transazioni:** Il validatore seleziona un insieme di transazioni dal mempool (pool di transazioni in attesa), tipicamente in base alle commissioni offerte per massimizzare il profitto.
2. **Esecuzione sequenziale:** Le transazioni vengono eseguite in ordine determinato dal validatore, applicando sistematicamente:
 - Trasferimenti di fondi tra account
 - Esecuzione del codice dei contratti attivati
 - Aggiornamento dello stato degli account coinvolti
3. **Determinismo garantito:** Il codice dei contratti è intrinsecamente deterministico - la stessa sequenza di transazioni, eseguita da qualsiasi validatore, produrrà identici risultati. Questa proprietà è fondamentale per il consenso distribuito.
4. **Inclusione della ricompensa:** Il validatore include la propria ricompensa per il blocco e le commissioni dalle transazioni.
5. **Calcolo dello stato risultante:** Viene calcolata la radice del nuovo world state trie, che rappresenta crittograficamente lo stato globale dopo l'applicazione delle transazioni nel blocco.

Una volta che un blocco è stato costruito e proposto, altri validatori selezionati verificano la sua correttezza, assicurandosi che:

- Tutte le transazioni siano valide e propriamente firmate
- L'esecuzione del codice sia corretta
- Lo stato finale calcolato sia accurato

Nel sistema Proof-of-Stake attuale, un validatore che attesta un blocco invalido può subire penalizzazioni economiche significative (slashing), perdendo parte o tutto lo stake depositato. Questo meccanismo di incentivi economici sostituisce l'investimento energetico del

Proof-of-Work, mantenendo la sicurezza del sistema attraverso razionalità economica piuttosto che costi computazionali.

7.2.4 Struttura dei Blocchi

La struttura dei blocchi Ethereum codifica tutte le informazioni necessarie per la validazione e l'incorporazione nella blockchain, con una progettazione orientata all'efficienza e alla verificabilità.

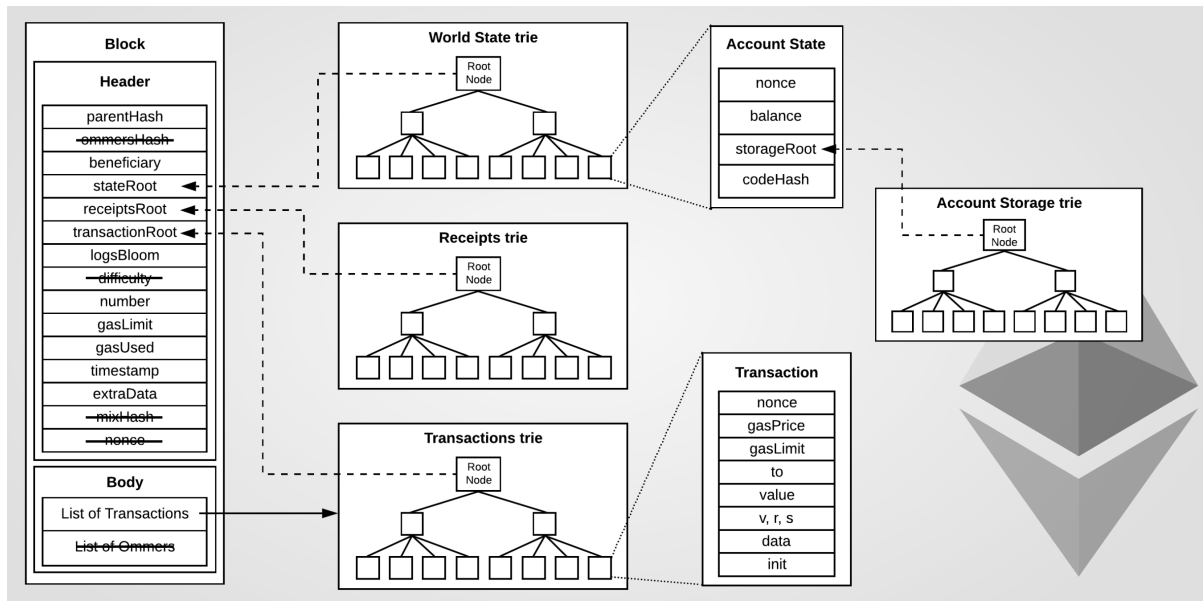


Figure 7.2: Struttura del blocco pre-merge.

Un blocco Ethereum (nella versione pre-Merge) comprende due componenti primari:

- **Header:** Contiene metadati essenziali e rappresentazioni crittografiche dello stato:
 - **parentHash:** Riferimento al blocco precedente, creando la catena
 - **beneficiary:** Indirizzo che riceve le ricompense e le commissioni
 - **stateRoot:** Radice del Merkle Patricia Trie che rappresenta lo stato globale
 - **transactionRoot:** Radice del trie delle transazioni
 - **receiptsRoot:** Radice del trie delle ricevute di transazione
 - **logsBloom:** Filtro Bloom per ricerca efficiente negli eventi emessi
 - **number:** Altezza del blocco nella blockchain
 - **gasLimit/gasUsed:** Limiti e utilizzo computazionale
 - **timestamp:** Momento di creazione del blocco
 - **extraData:** Campo per dati arbitrari
- **Body:** Contiene la lista effettiva delle transazioni incluse nel blocco.

Questa struttura è complementata da diverse strutture dati Merkle Patricia Trie interconnesse:

- **World State Trie:** Mappa indirizzi a stati di account
- **Transactions Trie:** Organizza le transazioni incluse nel blocco
- **Receipts Trie:** Memorizza le ricevute di esecuzione, inclusi log ed eventi
- **Storage Trie:** Per ciascun contratto, organizza il suo stato interno

Le radici di queste strutture sono incluse nell'header del blocco, permettendo la verifica crittografica di qualsiasi aspetto senza necessità di scaricare l'intero blocco o stato. Questa proprietà è fondamentale per l'implementazione di client leggeri (light clients) che possono validare transazioni con risorse limitate.

Dopo The Merge, la struttura dei blocchi è stata estesa per supportare il meccanismo Proof-of-Stake, aggiungendo campi relativi alle attestazioni dei validatori e alla randomness per la selezione dei propositori, ma i principi fondamentali restano invariati.

7.3 Il Sistema del Gas: Economia Computazionale

7.3.1 Problema dell'Halting e Soluzioni

L'introduzione di contratti Turing-completi in Ethereum ha sollevato una sfida fondamentale: come garantire che l'esecuzione del codice termini sempre in un tempo finito e prevedibile, permettendo alla rete di raggiungere un consenso deterministico sullo stato risultante.

Il problema fondamentale deriva dalla natura stessa dei linguaggi Turing-completi:

- La validazione di un blocco richiede l'esecuzione del codice nei contratti attivati dalle transazioni.
- In un sistema Turing-completo, è impossibile determinare a priori se un programma terminerà (Problema dell'Halting).
- Consentire l'esecuzione illimitata aprirebbe vulnerabilità critiche, permettendo attacchi DoS con loop infiniti che paralizzerebbero la rete.

Ethereum risolve questo dilemma introducendo il concetto di "gas", che costituisce una metrica economica per il consumo di risorse computazionali:

- Ogni operazione eseguita dalla Ethereum Virtual Machine (EVM) consuma una quantità predefinita di gas, proporzionale alla sua complessità computazionale.
- Il mittente di una transazione deve specificare un limite massimo di gas (gasLimit) e pagare anticipatamente per questa computazione potenziale.
- Se l'esecuzione consuma tutto il gas disponibile prima di completarsi, si verifica un errore "out-of-gas" e l'esecuzione termina, ripristinando lo stato ma trattenendo il gas consumato come commissione.

Questo meccanismo elegante trasforma un problema tecnico (garantire la terminazione) in uno economico: l'esecuzione può teoricamente proseguire indefinitamente, ma ha un

costo crescente che la rende economicamente insostenibile, disincentivando comportamenti malevoli o inefficienti.

È interessante notare che, contrariamente alla percezione comune, il problema fondamentale non risiede nei loop di per sé, ma nella prevedibilità del consumo di risorse. Un contratto potrebbe contenere loop potenzialmente lunghi ma comunque terminare entro limiti di gas ragionevoli se ben progettato.

7.3.2 Meccanismo Legacy del Gas

Prima dell'aggiornamento London (EIP-1559) a metà del 2021, Ethereum utilizzava un modello relativamente diretto per la gestione del gas e delle commissioni di transazione.

Il modello pre-London si basava su due parametri fondamentali specificati dal mittente:

- **gasLimit** (anche noto come **gasStart**): La quantità massima di gas che la transazione poteva consumare, rappresentando il limite superiore di computazione consentita.
- **gasPrice**: Il prezzo in wei che il mittente era disposto a pagare per unità di gas, determinando la priorità della transazione nel mempool. Il mercato stabiliva dinamicamente il prezzo ottimale, con servizi come ethgasstation.info che fornivano stime in tempo reale per diverse fasce di priorità.

Il consumo di gas seguiva regole precise per diversi tipi di operazioni:

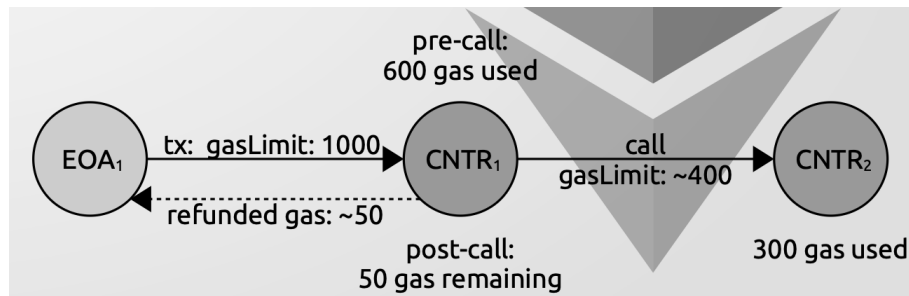
- **Istruzioni base**: Ogni operazione dell'EVM (somme, moltiplicazioni, hashing, ecc.) aveva un costo predefinito basato sulla complessità computazionale.
- **Accesso allo storage**: Operazioni di lettura/scrittura sulla memoria persistente (storage) comportavano costi significativamente maggiori rispetto a operazioni in memoria temporanea, riflettendo l'impatto a lungo termine sullo stato globale.
- **Rimborsi**: La liberazione di storage precedentemente occupato generava rimborsi parziali di gas, incentivando l'ottimizzazione dello spazio di stato.

Il flusso economico di una transazione seguiva un pattern standard:

- Il mittente prepagava il costo massimo potenziale ($\text{gasLimit} \times \text{gasPrice}$) all'invio della transazione.
- Durante l'esecuzione, il gas veniva detratto per ogni operazione.
- Gas non utilizzato veniva rimborsato al mittente al completamento dell'esecuzione.

Un aspetto critico erano gli errori out-of-gas, che potevano manifestarsi in due modalità:

- **Errori critici**: Terminazione dell'intera transazione con ripristino completo dello stato.
- **Errori gestiti**: In chiamate interne tra contratti, dove solo la sub-chiamata specifica falliva mentre la transazione principale poteva continuare.



La figura mostra un esempio di flusso di gas in una chiamata tra contratti: una transazione inizia con un gasLimit di 1000, ne utilizza 600 nella fase pre-chiamata, passa circa 400 al contratto chiamato, che ne consuma 300 e restituisce i 50 inutilizzati. Il calcolo preciso del gas necessario era una competenza critica per sviluppatori Ethereum, con stime inadeguate che portavano a transaction failures costose o inefficienti.

7.3.3 EIP-1559: Rivoluzione nel Mechanism Design

L'aggiornamento London di Ethereum a metà 2021 ha introdotto EIP-1559 (Ethereum Improvement Proposal 1559), una riforma radicale del meccanismo di gas che ha affrontato problemi strutturali del modello precedente: volatilità estrema dei prezzi del gas, incertezza nei tempi di inclusione, e user experience subottimale.

Il nuovo sistema si basa su un'architettura a tre componenti:

- **Base fee (baseFeePerGas):** Una commissione base dinamica determinata a livello di protocollo, che si autoregola in base alla congestione della rete. Se i blocchi sono più pieni del target (15M gas), la base fee aumenta; se sono meno pieni, diminuisce. Questo meccanismo crea un feedback automatico che stabilizza l'utilizzo della rete intorno al punto di equilibrio desiderato.

Crucialmente, questa commissione base viene "bruciata" (eliminata permanentemente dalla circolazione) anziché pagata ai validatori, introducendo un meccanismo deflazionistico nella tokenomics di Ethereum.

- **Priority fee (maxPriorityFeePerGas):** Una mancia opzionale specificata dal mittente, che viene pagata direttamente al validatore come incentivo per l'inclusione prioritaria. Questo componente rappresenta il vero mercato competitivo per l'accesso alla blockchain in periodi di alta domanda.
- **Max fee (maxFeePerGas):** Il limite superiore che il mittente è disposto a pagare complessivamente per unità di gas, formulato come:

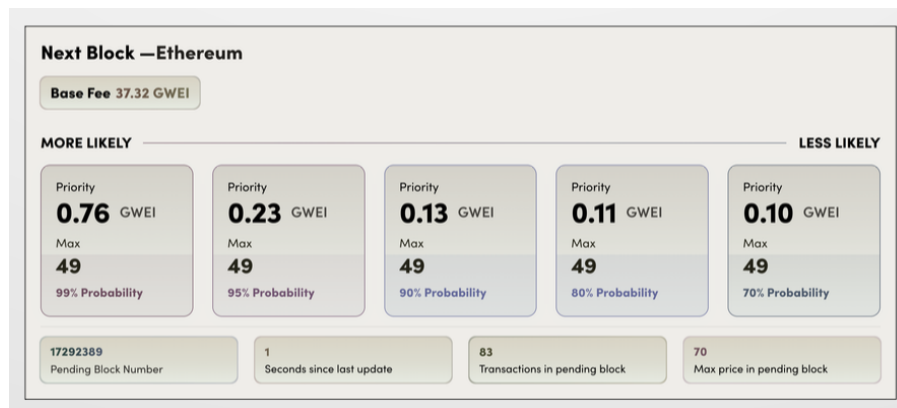
$$\text{commissione effettiva} = \min(\text{maxFeePerGas}, \text{baseFeePerGas} + \text{maxPriorityFeePerGas})$$

La differenza tra la commissione massima e quella effettivamente pagata viene rimborsata al mittente, proteggendolo da fluttuazioni impreviste della base fee tra la sottomissione della transazione e la sua inclusione.

Il gas limit continua a essere un parametro richiesto, specificando il limite massimo di computazione consentita per la transazione.

I benefici principali di EIP-1559 includono:

- **Prevedibilità migliorata:** La base fee auto-regolante riduce la volatilità estrema dei prezzi del gas.
- **UX ottimizzata:** Gli utenti possono specificare quanto sono disposti a pagare al massimo, con il sistema che automaticamente determina il prezzo ottimale.
- **Meccanismo deflazionistico:** La combustione della base fee può portare, in periodi di alta congestione, a una deflazione netta di ETH, potenzialmente aumentandone il valore nel lungo periodo.



EIP-1559 rappresenta uno dei casi più studiati di mechanism design nelle criptovalute, combinando teoria dei giochi, economia e incentivi in un sistema elegante che bilancia efficienza, usabilità e sostenibilità economica.

Chapter 8

Capitolo 8: Esempi di Transazioni Reali e Implicazioni

8.1 Anatomia di Transazioni Concrete

L'analisi di transazioni reali sulla blockchain Ethereum fornisce insight preziosi sul funzionamento pratico del sistema e sulle sue implicazioni. Gli esploratori blockchain come Etherscan permettono di esaminare in dettaglio ogni aspetto di queste transazioni.

The screenshot shows a transaction on Etherscan with the following details:

- Transaction Hash:** 0xb22ef906eaa55dea4547c877556ef4213e100541e60925857215cae7217b7cd2
- Status:** Success
- Block:** 12295329 (59 Block Confirmations)
- Timestamp:** 13 mins ago (Apr-23-2021 08:16:21 AM +UTC)
- From:** 0x6ec88a2cb932eb46dfa0280c0eadb93b6eca... (circled in red)
- To:** 0x085a849e3c32702da1a266dac54e52a5378c... (circled in blue)
- Value:** 0.20181911 Ether (\$435.24) (circled in yellow)
- Transaction Fee:** 0.0036036 Ether (\$7.77) (circled in green)
- Gas Price:** 0.0000001716 Ether (171.6 Gwei)
- Gas Limit:** 21,000
- Gas Used by Transaction:** 21,000 (100%)
- Nonce:** 10332 (Position 152)
- Input Data:** 0x

A modal window titled "Address" shows the state difference for three addresses:

Address	Before	After	State Difference
0x085a849e3c32702da1...	0 Eth	0.20181911 Eth	+ 0.20181911 (circled in yellow)
0x6ec88a2cb932eb46df...	30.42341784953860348 Eth	30.21799513953860348 Eth	- 0.20542271 (circled in green)
0x8595dd9e0438640b5e...	226.789913080338167395 Eth	226.793516680338167395 Eth	+ 0.0036036 (circled in green)

Below the modal, a diagram shows a "funding transfer" from a sender (key icon) to a receiver (key icon) with a large arrow pointing right.

8.1.1 Trasferimento di Fondi

Una transazione base di trasferimento evidenzia il flusso economico fondamentale:

- L'indirizzo mittente vede il proprio saldo decrementato dell'importo trasferito più la commissione di gas.
- L'indirizzo destinatario vede un incremento corrispondente al valore trasferito.
- La visualizzazione della "State Difference" mostra esattamente le modifiche allo stato prima e dopo la transazione.

Le transazioni di trasferimento sono relativamente economiche in termini di gas poiché non richiedono l'esecuzione di codice contrattuale complesso.

8.1.2 Interazione con Contratti (ERC-20)

Le transazioni che interagiscono con token ERC-20 rappresentano uno dei casi d'uso più comuni. L'esempio mostra:

- Il campo "To" indirizza un contratto anziché un EOA
- I dati di input codificano la chiamata di funzione (in questo caso `transferFrom`)
- Nessun ETH viene trasferito direttamente (`Value: 0 ETH`), ma token definiti dal contratto vengono movimentati
- Il consumo di gas è significativamente maggiore rispetto a un semplice trasferimento

Queste transazioni evidenziano la natura duale di Ethereum come piattaforma sia monetaria che computazionale, dove il valore può essere rappresentato non solo da ETH nativo ma anche da token definiti contrattualmente.

8.1.3 Transazioni Fallite (Out-of-Gas)

L'esempio di una transazione fallita per out-of-gas illustra le implicazioni economiche degli errori:

- La transazione consuma quasi tutto il gas allocato (98.41%) senza completare l'esecuzione
- Lo stato pre-transazione viene ripristinato, senza modifiche permanenti ai saldi o allo stato dei contratti
- Nonostante il fallimento, la commissione di gas viene comunque addebitata completamente per le risorse computazionali utilizzate

Questi fallimenti evidenziano l'importanza di una corretta stima del gas necessario e della comprensione del comportamento dei contratti con cui si interagisce.

8.2 Vulnerabilità ed Errori Celebri

La natura immutabile e automatizzata degli smart contracts ha portato a incidenti significativi che hanno evidenziato le sfide uniche di questa tecnologia.

8.2.1 The DAO Hack (2016)

The DAO (Decentralized Autonomous Organization) rappresenta uno dei casi più emblematici:

- Una vulnerabilità di "reentrancy" nel contratto ha permesso a un attaccante di drenare circa 50 milioni di dollari in ETH
- L'incidente ha portato a una controversa hard fork che ha diviso la blockchain in Ethereum (ETH) ed Ethereum Classic (ETC)

- Questo evento ha sollevato questioni fondamentali sulla governance della blockchain e sul principio "code is law"

La vulnerabilità di reentrancy sfruttata nell'attacco a The DAO è particolarmente istruttiva. Il problema derivava dalla sequenza di esecuzione nelle funzioni di prelievo:

1. Il contratto controllava se l'utente aveva diritto ai fondi
2. Inviava i fondi all'utente (chiamando una funzione esterna)
3. Aggiornava il saldo interno

L'attaccante ha creato un contratto malevolo la cui funzione di fallback, attivata durante la ricezione dei fondi al punto 2, richiama ricorsivamente la funzione di prelievo prima che il saldo fosse aggiornato al punto 3. Questo ha permesso prelievi multipli senza una corrispondente riduzione del saldo registrato.

Questo tipo di vulnerabilità evidenzia la complessità del design di smart contracts sicuri e l'importanza del pattern "Checks-Effects-Interactions" che raccomanda di aggiornare sempre lo stato interno prima di effettuare chiamate esterne.

8.2.2 Altri Casi Notevoli

Numerosi altri incidenti hanno evidenziato diverse classi di vulnerabilità:

- **Parity Multi-Sig Wallet:** Una vulnerabilità in un contratto di libreria condivisa ha portato al congelamento permanente di circa 150 milioni di dollari in ETH nel 2017, quando un utente inesperto ha accidentalmente preso possesso della libreria e successivamente l'ha distrutta.
- **Integer Overflow/Underflow:** Vulnerabilità matematiche in contratti che non verificavano adeguatamente i limiti delle operazioni aritmetiche, permettendo manipolazioni del saldo.
- **Front-running:** Attacchi basati sul riordino delle transazioni nel mempool, dove i miner o altri attori possono osservare transazioni pendenti e inserire le proprie in anticipo per trarne vantaggio.
- **Access Control Flaws:** Implementazioni inadeguate dei meccanismi di controllo dell'accesso, permettendo a entità non autorizzate di eseguire funzioni privilegiate.

La storia di questi incidenti ha portato allo sviluppo di strumenti di analisi statica, pratiche di audit formale, e pattern di programmazione difensiva che sono ora standard nell'ecosistema Ethereum.

8.3 Implicazioni Filosofiche e Legali

Gli errori nei smart contracts sollevano questioni etiche e legali complesse che vanno oltre il dominio puramente tecnico.

8.3.1 Code is Law?

Il principio "code is law", spesso citato nell'ecosistema blockchain, suggerisce che il codice eseguito sulla blockchain costituisce l'ultima autorità, indipendentemente dalle intenzioni dei suoi creatori o dalle aspettative degli utenti.

Questa visione presenta diverse implicazioni problematiche:

- **Definizione di "bug" vs "feature"**: Se un'azione rispetta le regole codificate in un contratto ma contravviene all'intento degli sviluppatori, costituisce un bug o semplicemente uno sfruttamento creativo delle regole?
- **Confini della legalità**: Considerando che molti di questi "attacchi" sfruttano semplicemente la logica codificata nei contratti, può essere considerato un crimine se il codice stesso lo permette?
- **Remediation**: Come gestire gli errori una volta scoperti, dato che l'immutabilità è una proprietà fondamentale della blockchain?

Il caso The DAO ha rappresentato un punto di svolta in questo dibattito. La decisione di implementare una hard fork per recuperare i fondi ha diviso la comunità tra:

- Sostenitori dell'immutabilità assoluta (che hanno continuato a supportare Ethereum Classic)
- Pragmatisti che hanno riconosciuto la necessità di interventi eccezionali in casi estremi (che hanno seguito Ethereum)

8.3.2 Responsabilità Giuridica

La natura decentralizzata degli smart contracts complica significativamente l'applicazione dei framework legali tradizionali:

- **Giurisdizione**: Quale sistema legale ha autorità su transazioni e contratti che esistono su una rete globale distribuita?
- **Responsabilità degli sviluppatori**: In quale misura gli sviluppatori di smart contracts possono essere ritenuti responsabili per vulnerabilità o comportamenti non intenzionali del codice?
- **Applicabilità legale**: Come possono essere fatte rispettare sentenze legali tradizionali in un sistema progettato per essere resistente alla censura?

Diverse giurisdizioni stanno sviluppando approcci differenti a queste questioni, con soluzioni che variano dall'integrazione degli smart contracts nei framework legali esistenti alla creazione di nuove categorie legali specifiche per la tecnologia blockchain.

8.3.3 Evoluzioni di Governance

In risposta alle sfide sollevate da questi incidenti, l'ecosistema Ethereum ha sviluppato diversi meccanismi di governance e sicurezza:

- **Timelocks e multisig**: Implementazioni tecniche che introducono ritardi deliberati o requisiti di approvazione multipla per azioni critiche.

- **Proxy patterns:** Architetture contrattuali che separano la logica di business (potenzialmente aggiornabile) dallo storage permanente, permettendo aggiornamenti con preservazione dello stato.
- **Processi di audit:** Procedure standardizzate di revisione del codice da parte di esperti per identificare vulnerabilità prima del deployment.
- **Bug bounty:** Programmi che incentivano la scoperta e la segnalazione responsabile di vulnerabilità.

Questi sviluppi rappresentano un bilanciamento pragmatico tra l'ideale dell'immutabilità e la realtà pratica della fallibilità del software, creando forme di "governabilità pianificata" che preservano i principi fondamentali della decentralizzazione ma riconoscono la necessità di meccanismi di correzione.